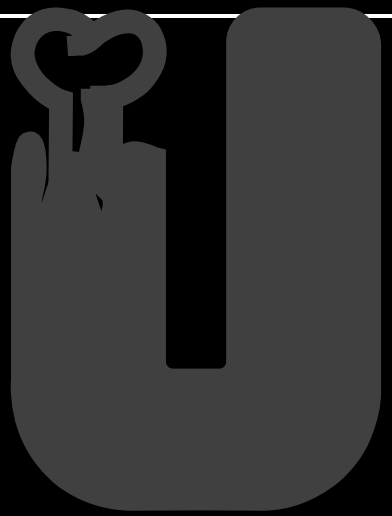
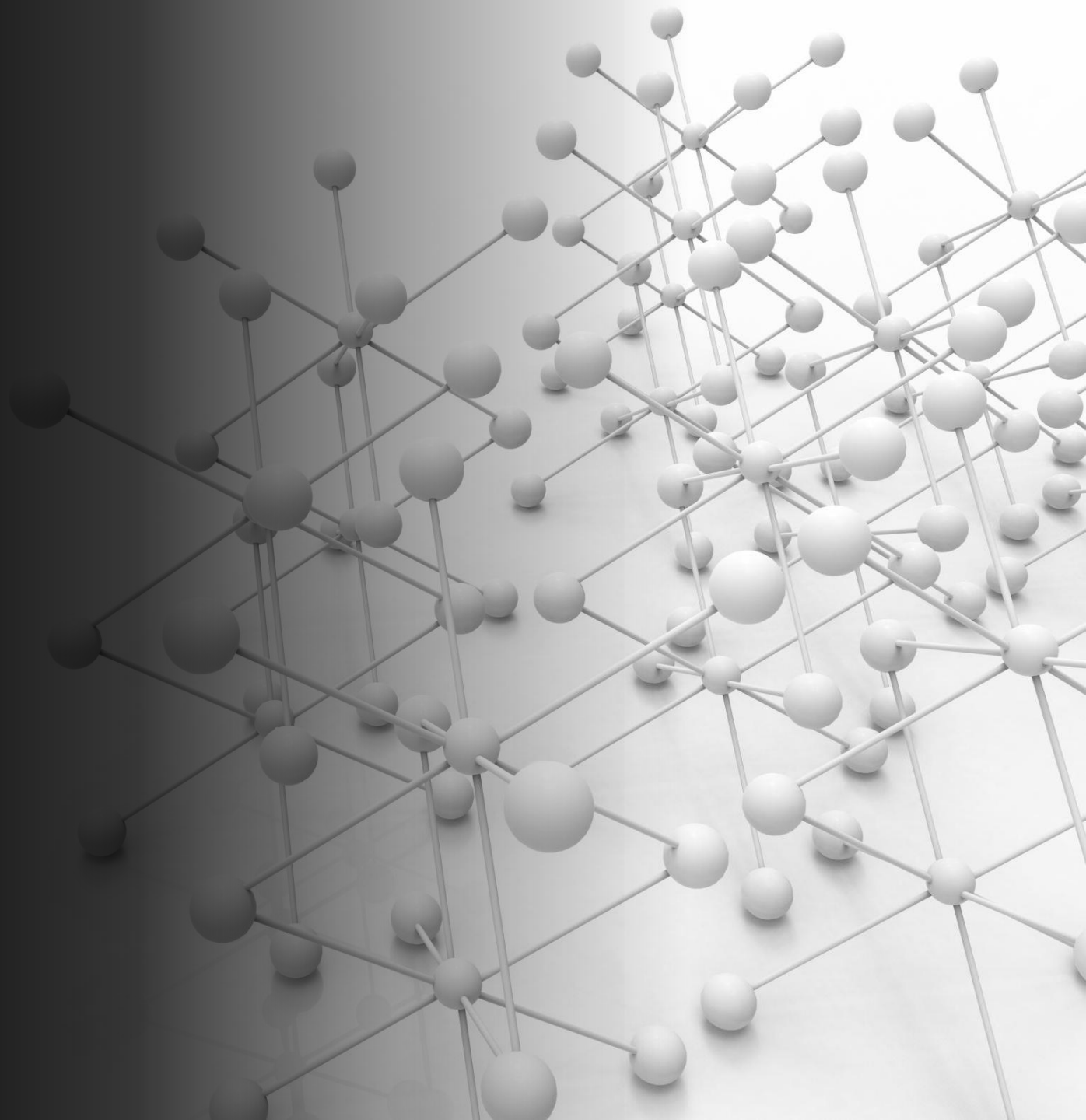
U6 – Grafos

GitHub - Jibaru/data-structure-and-algorithms: Algoritmos y estructura de datos en C++

Grafos Representación de los Grafos





Grafos (Graph)

U6



7 GRAFOS

01
7.1. CONCEPTOS Y DEFINICIONES
7.1.1. Grado de entrada, grado de salida de un nodo
7.1.2. Camino
7.1.3. Tipo Abstracto de Datos Grafo

02
7.2. REPRESENTACIÓN DE LOS GRAFOS
7.2.1. Matriz de adyacencia
7.2.2. Matriz de adyacencia: class GrafoMatriz

03
7.3. LISTAS DE ADYACENCIA

04
7.4. RECORRIDO DE UN GRAFO
7.4.1. Recorrido en anchura
7.4.2. Recorrido en profundidad
7.4.3. Implementación: clase RecorreGrafo

05
7.5. CONEXIONES EN UN GRAFO
7.5.1. Componentes conexas de un grafo
7.5.2. Componentes fuertemente conexas de un grafo

06
7.6. MATRIZ DE CAMINOS. CIERRE TRANSITIVO
7.6.1. Matriz de caminos y cierre transitivo

07
7.7. ORDENACIÓN TOPOLÓGICA
7.7.1. Algoritmo para obtener una ordenación topológica
7.7.2. Implementación del algoritmo de ordenación topológica

08
7.8. MATRIZ DE CAMINOS: ALGORITMO DE WARSHALL
7.8.1. Implementación del algoritmo de Warshall

09
7.9. CAMINOS MÍNIMOS CON UN SOLO ORIGEN: ALGORITMO DE DIJKSTRA
7.9.1. Algoritmo de Dijkstra
7.9.2. Codificación del algoritmo de Dijkstra

10
7.10. TODOS LOS CAMINOS MÍNIMOS: ALGORITMO DE FLOYD
7.10.1. Codificación del algoritmo de Floyd

11
7.11. ÁRBOL DE EXPANSIÓN DE COSTE MÍNIMO
7.11.1. Algoritmo de Prim
7.11.2. Codificación del algoritmo de Prim
7.11.3. Algoritmo de Kruscal

Grafos| Graph
Introducción.

Graph 1

Grafos| C++
Introducción.

Este unidad introducirá al Alumno a conceptos matemáticos importantes denominados grafos que tienen aplicaciones en campos tan diversos como sociología, química, geografía, ingeniería eléctrica e industrial, etc. **Los grafos se estudian como estructuras de datos o tipos abstractos de datos**. **Estudiaremos las definiciones relativas a los grafos y la representación de los grafos en la memoria del ordenador.** Investigaremos dos formas tradicionales de implementación de grafo, **matriz de adyacencia y listas de adyacencia**. También se estudian operaciones importantes y algoritmos de grafos que son significativos en informática. **Existen numerosos problemas que se pueden modelar en términos de grafos**. Ejemplo de ello es la planificación de las tareas que completan un proyecto, encontrar las rutas de menor longitud entre dos puntos geográficos, calcular el camino más rápido en un transporte, determinar el flujo máximo que puede llegar desde una fuente a, por ejemplo, una urbanización. La resolución de estos problemas requiere examinar todos los nodos y las aristas del grafo que representa al problema.

Los algoritmos imponen implícitamente un orden en estas visitas: el nodo más próximo o las aristas más cortas, y así sucesivamente; no todos los algoritmos requieren un orden concreto en el recorrido del grafo. Según lo anterior, **se estudian en esta unidad el concepto de ordenación topológica, los problemas del camino más corto, junto con el concepto de árbol de expansión de coste mínimo. Estos algoritmos han sido desarrollado por grandes investigadores y en su honor se conocen por su nombre los algoritmos de Dijkstra, Warshall, Prim, Krusca**l, Ford-Fulkerson ...

Grafos | Graph

Conceptos y Definiciones

Graph

Grafos Conceptos y Definiciones

CONCEPTOS Y DEFINICIONES

Un grafo G agrupa entes físico o conceptuales y las relaciones entre ellos. Por tanto, ***un grafo está formado por un conjunto de vértices o nodos V, que representan a los entes, y un conjunto de arcos A, que representan las relaciones entre vértices***. Se representa con el par G = (V, A). La Figura 18.1 muestra un grafo formado por los vértices V = {1,4,5,7,9} y el conjunto de arcos A = {(1,4), (4,1), (5,1), (1, 5), (7,9), (9,7), (7,5), (5,7), (4,9), (9,4)}

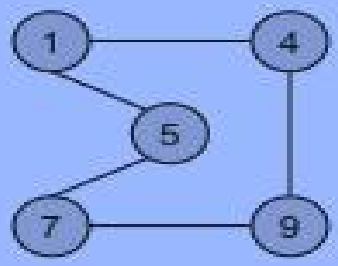


Figura 18.1. Grafo no dirigido.

Un arco o arista representa una relación entre dos nodos. Esta relación, al estar formada por dos nodos, se representa por (u, v) siendo u, v el par de nodos. El grafo es no dirigido si los arcos están formados por pares de nodos no ordenados, no apuntados; se representa con un segmento uniendo los nodos, u — v. El grafo de la Figura 18.1 es no dirigido. Un grafo es dirigido, también denominado digrafo, si los pares de nodos que forman los arcos son ordenados; se representan con una flecha que indica la dirección de la relación, u l v. El grafo de la Figura 18.2 que consta de los vértices, V = {C, D, E, F, H}, y de los arcos A = {(C, D,), (D, F), (E, H), (H, E), (E, C)} forman el grafo dirigido G = {V, A

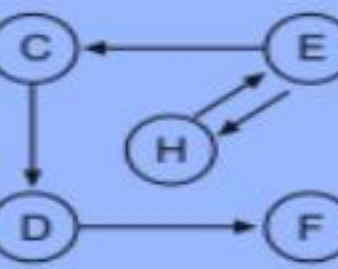


Figura 18.2. Grafo dirigido.

Dado el arco (u, v) de un grafo, se dice que los vértices u y v son adyacentes. Si el grafo es dirigido, el vértice u es adyacente a v, y v es adyacente de u. En los modelos realizados con grafos, a veces, una relación entre dos nodos tiene asociada una magnitud, denominada factor de peso, en cuyo caso se dice que es un grafo valorado. Por ejemplo, los pueblos que forman una comarca junto a la relación entre un par de pueblos de estar unidos por un camino: esta relación tiene asociado el factor de peso, que es la distancia en kilómetros. La Figura 18.3 muestra un grafo valorado en el que cada arco tiene asociado un peso que es la longitud entre dos nodos.

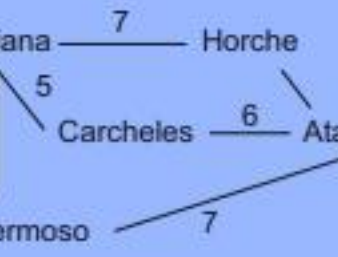


Figura 18.3. Grafo no dirigido valorado.

Grafos Conceptos y Definiciones

Grado de entrada, grado de salida de un nodo

El grado es una cualidad que se refiere a los nodos de un grafo. En un grafo no dirigido el grado de un nodo v, grado(v), es el número de arcos que contiene a v. En un grafo dirigido se distingue entre grado de entrada y grado de salida; grado de entrada de un nodo v, gradent(v), es el número de arcos que llegan a v; grado de salida de v, gradsal(v), es el número de arcos que salen de v. Así, en el grafo no dirigido de la Figura 18.3, (Lupiana) = 3. En el grafo dirigido de la Figura 18.2, Gradient (D) = 1 , y el Gradsal(D) = 1



Figura 18.3. Grafo no dirigido valorado.

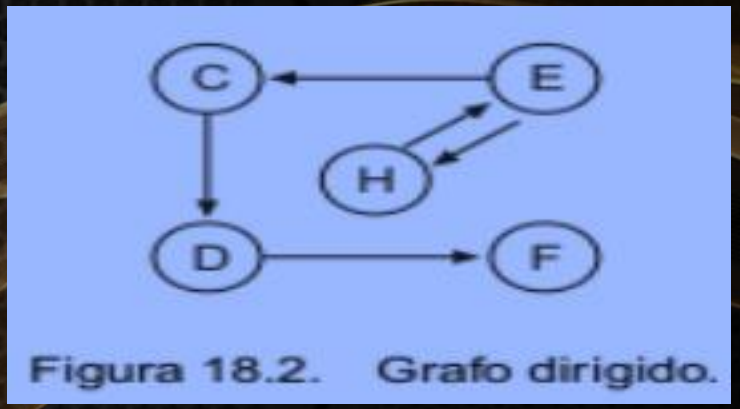


Figura 18.2. Grafo dirigido.

Camino

Un camino P de longitud n desde el vértice v0 a vn en un grafo G, es la secuencia de n + 1 vértices v0, v1, v2,..., vn tal que (vi , vi+1) ↓ A(arcos)para 0 a i n. Matemáticamente el camino se representa por P = (v0, v1, v2, ..., vn). En la Figura 18.4 se pueden encontrar más de un camino; por ejemplo, P1 = (4, 6, 9, 7) es un camino de longitud 3. Otro de los caminos es P2 = (10, 11), que tiene de longitud 1. Por tanto, se puede afirmar que la longitud del camino es el número de arcos que lo forman.

En el grafo valorado de la Figura 18.3, el camino (Lupiana, Valfermoso, Atanzón) tiene de longitud 12 + 7 = 19. En algunos grafos se dan arcos desde un vértice a sí mismo, (v, v); entonces el camino v l v es un bucle. Normalmente, en los grafos no hay nodos relacionados con sí mismo, no es frecuente encontrarse grafos con bucles.

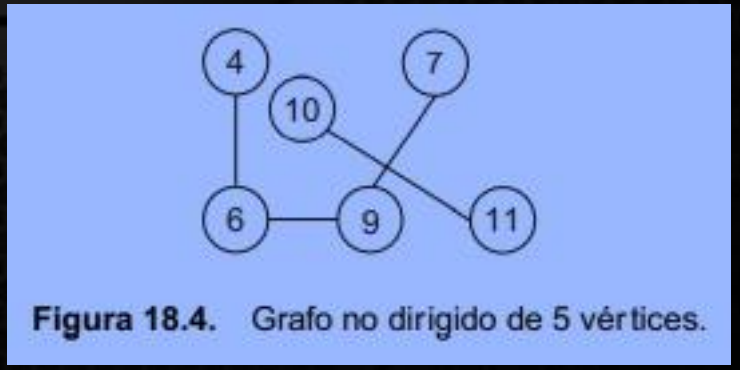


Figura 18.4. Grafo no dirigido de 5 vértices.

Grafos Conceptos y Definiciones

Camino (cont…)

Un camino P = (v0, v1, v2, ..., vn) es simple si todos los nodos que forman el camino son distintos, pudiendo ser iguales los extremos del camino v0, vn . En el grafo de la Figura 18.4 el camino P1 y P2 son caminos simples. En un grafo dirigido, un ciclo es un camino simple cerrado. Por tanto, un ciclo empieza y termina en el mismo nodo, v0 = vn, y, además, debe tener más de un arco. Un grafo dirigido sin ciclos (acíclico) se acostumbra a denominar mediante la abreviatura GDA (Grafo Dirigido Acíclico). La Figura 18.5 muestra un grafo dirigido en el que los vértices (A, E, B, F, A) forman un ciclo de longitud 4. En general, un ciclo de longitud k se denomina k-ciclo.

Un grafo no dirigido es conexo si existe un camino entre cualquier par de nodos que forman el grafo. En el caso de un grafo dirigido con esta propiedad se dice que es fuertemente conexo. Además, un grafo completo es aquél que tiene un arco para cualquier par de vértices.



Figura 18.6. *a*) Grafo conexo. *b*) Grafo fuertemente conexo.

Tipo Abstracto de Datos Grafo

Un grafo consta de un conjunto de objetos o elementos junto a sus relaciones. Es preciso definir las operaciones básicas para construir la estructura y, en general, modificar sus elementos. En definiva, especificar el tipo abstracto de datos grafo. Ahora se definen operaciones básicas, a partir de las cuales se construye el grafo. Su realización depende de la representación elegida (**matiz de adyacencia, o listas de adyacencia**). Algunas Operaciones:

arista (u, v): Añade el arco o arista(u, v) al grafo.
aristaPeso(u,v, w): Para un grafo valorado, añade el arco(u,v) al grafo y el coste del arco, w.
borraArco(u,v): Elimina del grafo el arco(u, v).
adyacente(u,v): Operación que devuelve cierto si los vértices u, v forman un arco.
nuevoVertice(u): Añade el vértice u al grafo G.
borraVértice(u): Elimina el vértice u del grafo G.

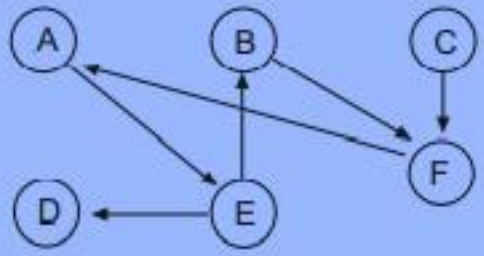


Figura 18.5. Grafo dirigido con ciclos.

Grafos | Graph

Representación de los Grafos

Grafos Representación de los Grafos

Para trabajar con los grafos y aplicar algoritmos que permitan encontrar propiedades entre los nodos hay que pensar en su representación. Cómo representar un grafo en memoria interna, qué tipos o estructuras de datos se deben utilizar para considerar los nodos y los arcos.

Matriz de Adyacencia y Listas de Adyacencia.

Una primera simplificación que se hace es considerar a los vértices o nodos como números consecutivos, empezando por el vértice 0. Hay que tener en cuenta que se tiene que representar un número (finito) de vértices y de arcos que unen dos vértices. Se puede elegir una representación secuencial, mediante un array bidimensional, conocida como **Matriz de adyacencia**;

o bien, una representación dinámica, mediante una estructura multienlazada, denominada **Listas de adyacencia**. La elección de una representación u otra depende del tipo de grafo y de las operaciones que se vayan a realizar sobre los vértices y arcos.

Para un grafo denso (tiene la mayoría de los arcos posibles) lo mejor es utilizar una matiz de adyacencia.
Para un grafo disperso (tiene, relativamente, pocos arcos) se suele utilizar listas de adyacencia que se ajustan al número de Arcos

Grafos| Graph

Graph 3

Representación de los Grafos
Matriz de Adyacencia
Listas de Adyacencia

Grafos | Graph

Matriz de Adyacencia

Graph 3.1

Grafos Representación de los Grafos Matriz de adyacencia

La característica más importante de un grafo, que distingue a un grafo de otro, es el conjunto de pares de vértices que están relacionados, o que son adyacentes. Por ello, la forma más sencilla de representar un grafo es mediante una matriz, de tantas filas / columnas como nodos, que permite modelar fácilmente esa cualidad. Sea G = (V, A) un grafo de n nodos, siendo V = {v0, v1, ..., vn-1} el conjunto de nodos, y A = {(vi, vj)} el conjunto de arcos. Los nodos se pueden representar por números consecutivos de 0 a n - 1. La representación de los arcos se hace con una matriz A de nXn elementos, denominada matriz de adyacencia, tal que todo elemento aij puede tomar los valores:

$$a_{ij} = \begin{cases} 1 & \text{si hay un arco } (v_i, v_j) \\ 0 & \text{si no hay arco } (v_i, v_j) \end{cases}$$

La Figura 18.7 representa un grafo dirigido, suponiendo que el orden de los vértices es el de la secuencia {D, F, K, L, R}, entonces la matriz de adyacencia:

$$A = \begin{vmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{vmatrix}$$

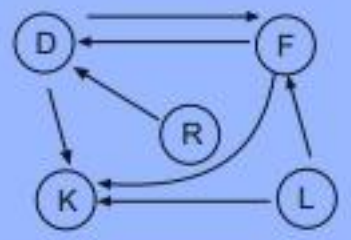


Figura 18.7. Grafo dirigido con los vértices {D, F, K, L, R}.

El grafo de la Figura 18.8 es no dirigido, está formado por 5 vértices. La matriz de adyacencia:

$$A = \begin{vmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{vmatrix}$$

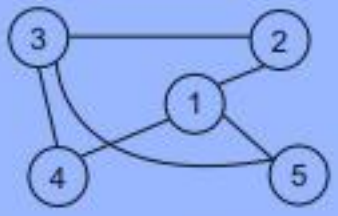


Figura 18.8. Grafo no dirigido con 5 vértices.

Grafos Representación de los Grafos

Matriz de adyacencia (Valorada)

Los grafos que modelan problemas en los que un arco tiene asociado una magnitud, un factor de peso, también se representan mediante una matriz de tantas filas / columnas como nodos. Ahora un elemento cualquiera, ahi representa el coste o factor de peso del arco (vi, vj). Un arco que no existe se puede representar con un valor imposible, por ejemplo, infinito; también, se puede representar con el valor 0, dependiendo de que para el grafo el 0 no pueda ser un factor de peso significativo de un arco. A esta matriz también se la denomina matriz valorada. El grafo valorado de la Figura 18.9 se corresponde con un grafo dirigido con factor de peso. Si se supone que los vértices se numeran en el orden de V = {Alicante, Barcelona, Cartagena, Murcia, Reus}, la matriz de pesos P en la que se representa con 0 la no existencia de arco

$$P = \begin{vmatrix} 0 & 4 & 5 & 0 & 0 \\ 0 & 0 & 3 & 6 & 3 \\ 3 & 0 & 0 & 0 & 0 \\ 5 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{vmatrix}$$

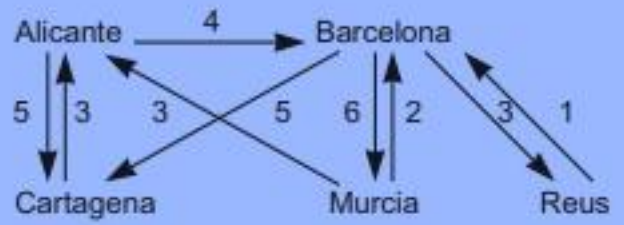


Figura 18.9. Grafo dirigido con factor de peso.

A considerar

La matriz de adyacencia representa los arcos, las relaciones entre un par de nodos de un grafo. Es una matriz de unos y ceros, que indican que dos vértices son adyacentes o no. En un grafo valorado, cada elemento representa el peso de la arista, y por ello se la denomina *matriz de pesos*.

Grafos Representación de los Grafos

Matriz de adyacencia: class Vertice (GrafoMatriz)

A todos los nodos del grafo se les da un nombre (string), y se le asigna un número, a partir de 0, en la matriz de adyacencia. La clase Vértice representa un nodo del grafo. Sus tres constructores sobrecargados son los encargados de inicializar los dos atributos protegidos nombre y numVertice. El primero de ellos lo hace por defecto; el segundo inicializa el nombre y pone como número de vértice –1; el tercero inicializa el número de vértice y el nombre. El resto de los métodos de la clase Vertice realizan las funciones de acceder o modificar cada uno de los atributos. La función miembro igual(), decide si el nombre de un vértice que recibe como parámetro coincide con el del objeto actual

```
class Vertice {
  protected:

    string nombre;
    int numVertice;

public:
    Vertice() {
    }
    Vertice(string x) { // inicializa el nombre y pone el número de vértice e -1
    nombre = x;
    numVertice = -1;
    }
  Vertice(string x, int n) { // inicializa el nombre y el número de vértice
    nombre = x;
    numVertice = n;
    }
  string OnomVertice() { // retorna el nombre del vértice
    return nombre;
    }
  void PnomVertice(string nom) { // cambia el nombre del vértice
    nombre = nom;
    }
  bool igual(Vertice n) { // decide entre la igualdad de nombres
    return nombre == n.nombre;
    }
  void PnumVertice(int n) { // cambia el número del vértice
    numVertice = n;
    }
  int OnumVertice() { // retorna el número del vértice
    return numVertice;
    }
};
```

Grafos Representación de los Grafos

Matriz de adyacencia: class GrafoMatriz

La clase GrafoMatriz contiene como atributos protegidos su tamaño (número máximo de vértices), el número de vértices actual, la matriz de adyacencia de dimensión variable y un array dimensionable que almacena los distintos Vértices. Además, contiene todos los métodos públicos necesarios para acceder o modificar cada uno de sus atributos. Posteriormente, se añadirán métodos que implementan operaciones comunes.

```
class GrafoMatriz {
  protected:
    int maxVerts; // máximo numero de vértices
    int numVerts; // número de vértices actual
    Vertice * verts; // array de vértices
    int ** matAd; // matriz de adyacencia
  public:
    // métodos públicos de la clase GrafoMatriz
    private:
    // métodos privados de la clase GrafoMariz
};
```

El constructor sin argumentos crea la matriz de adyacencia para un máximo de vértices preestablecido que en este caso es 1; el otro constructor tiene un argumento con el máxim

```
typedef int * pint;
// para el dimensionamiento de la matriz

GrafoMatriz::GrafoMatriz(int mx) {

  maxVerts = mx;
  verts = new Vertice[mx]; // vector de vértices
  matAd = new pint[mx]; // vector de punteros
  numVerts = 0;
  for (int i = 0; i < mx; i++)
    matAd[i] = new int[mx]; // matriz de adyacencia
}

GrafoMatriz::GrafoMatriz() {
  maxVerts = 1;
  GrafoMatriz(maxVerts);
}
```

Las funciones miembro públicas encargadas de obtener el número de vértices o de cambiar el número de vértices de la clase GrafoMatriz son

```
int OnumeroDeVertices(){return numVerts;}
void PnumeroDeVertices(int n){numVerts = n;}
```

Grafos Representación de los Grafos

Matriz de adyacencia: class GrafoMatriz

Añadir un vértice

La operación nuevoVertice() recibe el nombre de un vértice del grafo, comprueba si el nombre recibido se encuentra en la lista de vértices incrementando en caso negativo el número de vértices y asignándolo a la lista. La comprobación de si el nombre está en lista de vértices es realizado por el método numVertice().

```
void GrafoMatriz::nuevoVertice(string nom) {
  bool esta = numVertice(nom) >= 0;
  if (!esta) {
    Vertice v = Vertice(nom, numVerts);
    verts[numVerts++] = v; // se asigna a la lista.
    // No se comprueba que sobrepase el máximo
  }
}
```

numVertice() es un método público de la clase, busca el vértice en el array, retornando -1 si no lo encuentra:

```
int GrafoMatriz::numVertice(string v) {
  int i;
  bool encontrado = false;
  for (i = 0;(i < numVerts) && !encontrado;) {
    encontrado = verts[i].igual(v);
    if (!encontrado) i++;
  }
  return (i < numVerts) ? i : -1;
}
```

Para realizar la entrada completa del grafo en memoria, primero se asignan los vértices y, a continuación, sus arcos. El tiempo de esta operación depende de la densidad del grafo, si se considera un grafo denso el tiempo de ejecución es cuadrático, O(n2), siendo n el número de nodos

Grafos Representación de los Grafos

Añadir un arco

El método sobrecargado y público nuevoArco() recibe el nombre de los dos vértices que forman el arco y busca en el array, el número de vértice asignado a cada uno de ellos marcando la matriz de adyacencia

```
void GrafoMatriz::nuevoArco(string a, string b) {
  int va, vb;
  va = numVertice(a);
  vb = numVertice(b);
  if (va < 0 || vb < 0)
     throw "Vértice no existe";
  matAd[va][vb] = 1;
}
```

Otra versión sobrecargada del método público recibe, directamente, los números de los vértices que forman el arco y marca la posición de la matriz.

```
void GrafoMatriz::nuevoArco(int va, int vb) {
  if (va < 0 || vb < 0 || va > numVerts || vb > numVerts)
    throw "Vértice no existe";
  matAd[va][vb] = 1;
}
```

Este método tiene un tercer argumento para grafos valorados, que representa el factor de peso del arco que se asigna a la matriz. Por ejemplo, en el caso anterior, la versión de la función miembro es:

```
void GrafoMatriz::nuevoArco(int va, int vb, int valor) {
  if (va < 0 || vb < 0 || va > numVerts || vb > numVerts)
    throw "Vértice no existe";
  matAd[va][vb] = valor;
}
```

Grafos Representación de los Grafos

GrafoMatriz::adyacent

Determina si dos vértices, v1 y v2, forman un arco. Sencillamente, si el elemento de la matriz de adyacencia es 1. Se escriben dos versiones de la función miembro pública adyacente(). La primera tiene los nombres de los vértices como parámetro y la segunda los números de vértice.

```
bool GrafoMatriz::adyacente(string a, string b)
{
 int va, vb;
 va = numVertice(a);
 vb = numVertice(b);
 if (va < 0 || vb < 0) throw "Vértice no existe";
 return matAd[va][vb] == 1;
}
bool GrafoMatriz::adyacente(int va, int vb)
{
 if (va < 0 || vb < 0 || va >= numVerts || vb >= numVerts)
 throw "Vértice no existe";
 return matAd[va][vb] == 1;
}
```

GrafoMatriz::Ovalor **Valor de la matriz de adyacencia**

El método público sobrecargado Ovalor() se encarga de retornar el valor almacenado en la matriz de adyacencia para los vértices que recibe como parámetro, identificados con el nombre del vértice o con el número asignado. Es muy similar al método adyacente(), excepto que en este caso se retorna el propio contenido de la matriz, por lo que puede servir para grafos valorados. Hay que tener en cuenta que esta función es muy importante para el tratamiento de los algoritmos de grafos, ya que retorna el valor de las distintas entradas de la matriz de adyacencia o de pesos para grafos no valorados o valorados respecivamene.

```
int GrafoMatriz::Ovalor(string a, string b) {
  int va, vb;
  va = numVertice(a);
  vb = numVertice(b);
  if (va < 0 || vb < 0) throw "Vértice no existe";
  return matAd[va][vb];
}
int GrafoMatriz::Ovalor(int va, int vb) {
  if (va < 0 || vb < 0 || va >= numVerts || vb >= numVerts)
    throw "Vértice no existe";
  return matAd[va][vb];
}
```

Grafos Representación de los Grafos

`GrafoMatriz::Pvalor`
Modificar el valor de la matriz de adyacencia

El método público Pvalor() modifica el valor de la posición de la matriz de adyacencia que recibe como parámetro, o bien mediante el nombre o bien mediante el número del vértice. La función miembro es interesante para usarla tanto en grafos valorados como no valorados ya que permite modificar una entrada de la matriz de pesos o de adyacencia

```
void GrafoMatriz::Pvalor(int va, int vb, int v) {
  if (va < 0 || vb < 0 || va >= numVerts || vb >= numVerts)
    throw "Vértice no existe";
  matAd[va][vb] = v;
}
void GrafoMatriz::Pvalor(char * a, char * b, int v) {
  int va, vb;
  va = numVertice(a);
  vb = numVertice(b);
  if (va < 0 || vb < 0) throw "Vértice no existe";
  matAd[va][vb] = v;
}
```

`Vertice::Overtice` **Vértice asociado a un número entero**

El método público Overtice() recibe como parámetro una identificación del vértice y retorna todos los atributos del vértice si está almacenado en el grafo. La importancia de esta función miembro radica en permitir obtener toda la información asociada a un vértice almacenada en el grafo, para su posterior consulta.

```
Vertice Overtice(int va)
{
 if (va < 0 || va >= numVerts)
 throw "Vértice no existe";
 else return verts[va]
}
```

`Pvertice` Resulta interesante el método público Pvertice() que permite modificar la información de un vértice, para el caso de que el vértice se identifique como un número entero

```
void Pvertice(int va, Vertice vert) {
  if (va < 0 || va >= numVerts)
    throw "Vértice no existe";
  else verts[va] = vert;
}
```

Grafos| Graph

Listas de Adyacencia

Graph 3.2

Grafos Representación de los Grafos LISTAS DE ADYACENCIA

La representación de un grafo con matriz de adyacencia no es eficiente cuando el grafo es poco denso, (disperso), es decir tiene pocos arcos y, por tanto, la matriz de adyacencia tiene muchos ceros. Para grafos dispersos la matriz de adyacencia ocupa el mismo espacio que si el grafo tuviera muchos arcos (grafo denso). Cuando esto ocurre, se elige la representación del grafo mediante listas enlazadas, denominadas listas de adyacencia. Las listas de adyacencia son una estructura multienlazada formada por una tabla directorio donde cada elemento de la tabla representa un vértice del grafo, del que emerge una lista enlazada con todos sus vértices adyacentes. Esto es, cada lista representa a los arcos con vértice origen el del nodo de la lista directorio, por eso se llama lista de adyacencia.

La Figura 19.10 representa a un grafo dirigido, la representación mediante listas de adyacencia se encuentra en la Figura 19.11. Si por ejemplo se analiza el vértice 5, es adyacente a los vértices 1, 2 y 4; por ello su lista de adyacencia consta de tres nodos, cada uno con el vértice destino que forma el arco. También se puede observar que el vértice 4 no es origen de ningún arco, como consecuencia la lista de adyacencia está vacía.

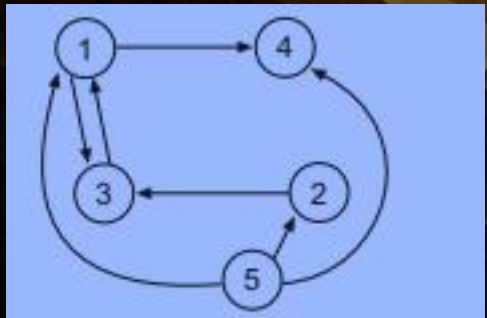


Figura 18.10. Gr afo dirigido.

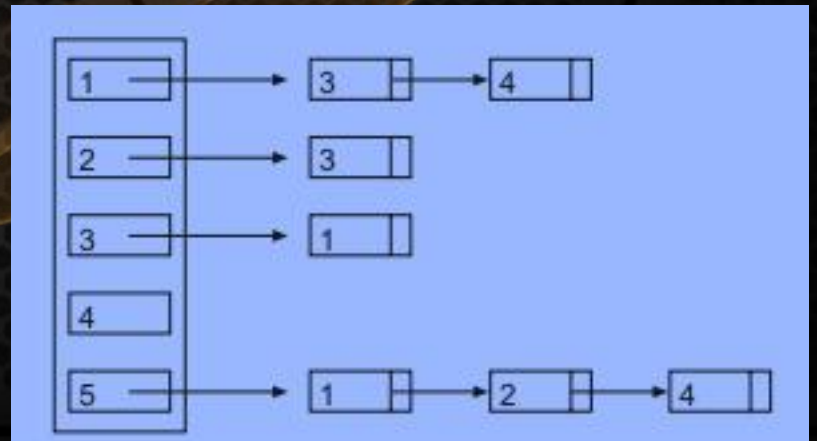


Figura 18.11. Listas de adyacencia del grafo 19.10.

Grafos| Graph

Recorrido de un Grafo

Grafos Representación de los Grafos

RECORRIDO DE UN GRAFO

En general, recorrer una estructura consiste en visitar (procesar) cada uno de los nodos a partir de uno dado; así se ha realizado el recorrido de un árbol en, por ejemplo, preorden partiendo del nodo raíz. De igual forma, recorrer un grafo consiste en visitar todos los vértices alcanzables a partir de uno dado. Muchos de los problemas que se plantean con los grafos exigen examinar las aristas de que consta y procesar los vértices del grafo. Sea V el conjunto de vértices del grafo, los algoritmos de recorrido de grafos parten de un nodo, v, y mantienen un conjunto de nodos marcados (procesados), W, que inicialmente es el nodo o vértice de partida, v. En cada pasada del algoritmo se retira un nodo, w, del conjunto W, se procesa y para cada una de las aristas que tengan como origen el vértice W, (W, U) su vértice adyacente, u, se añade a W si no está marcado. El algoritmo continúa hasta que el conjunto W queda vacío, en ese momento todo vértice no marcado no es accesible desde el nodo de partida v. Si se necesita un recorrido completo, se puede continuar desde cualquier vértice no marcado.

RECORRIDO DE UN GRAFO

Recorrido en Profundidad.

Recorrido en Anchura.

A tener en cuenta

Hay dos formas: de recorrer un grafo: *recorrido en profundidad y recorrido en anchura.* Si el conjunto de nodos *marcados* se trata como una cola, entonces el recorrido es en anchura; si se trata como una pila, el recorrido es en profundidad.

Grafos Representación de los Grafos RECORRIDO DE UN GRAFO

1. Recorrido en anchura

Se utiliza una **cola** como estructura en la que se mantienen los vértices marcados que se van a procesar posteriormente. El proceso de los elementos en una cola (primero en entrar primero en salir) hace que, a partir del vértice de partida, v, se procesen primero todos los vértices adyacentes a v, después los adyacentes de éstos que no estuvieran ya marcados o visitados, y así sucesivamente con los adyacentes de los adyacentes.

El orden de visitar los nodos, en el recorrido en anchura, se puede expresar de manera más concisa en estos pasos:

1. Marcar el vértice de partida v.
2. Meter en la cola el vértice de partida v.
3. Repetir los pasos 4 y 5 hasta que se cumpla la condición cola vacía.
4. Quitar nodo frente de la cola, w, visitar w.
5. Meter en la cola todos los vértices adyacentes a w que no estén marcados, a continuación marcar esos vértices.
6. Fin del recorrido

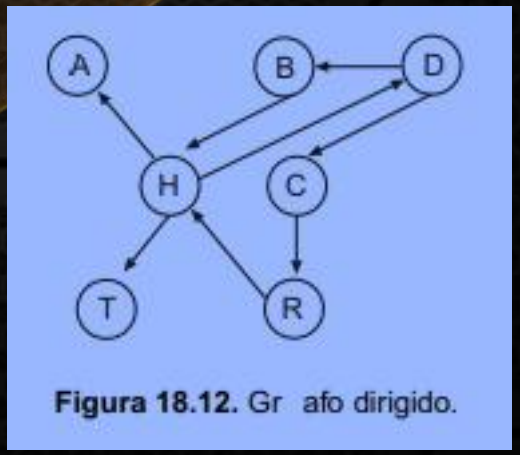


Figura 18.12. Gr afo dirigido.

Grafos Representación de los Grafos RECORRIDO DE UN GRAFO

1. Recorrido en anchura

Como ejemplo, se va a recorrer en anchura el grafo dirigido de la Figura 18.12. El recorrido se inicia a partir del nodo D, se marca y se mete en la cola. Ahora iterativamente, se quita el nodo frente, se procesa, se mete en la cola sus adyacentes no marcados y se marcan. Los sucesivos elementos de la cola se muestran en la Figura 18.13. En la columna de vértices procesados el vértice en negrita es el que se procesa en esa pasada, y en la cola los vértices en cursiva son los que se meten en la cola y son marcados.

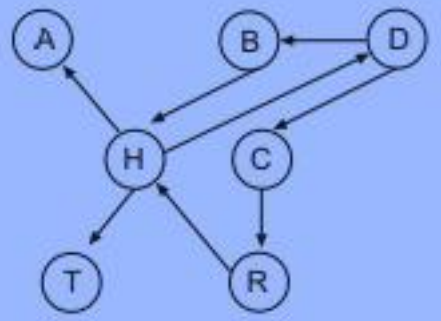


Figura 18.12. Gr afo dirigido.

COLA	Vértices procesados
D	
B *C*	**D**
C *H*	**B**
H *R*	**C**
R A *T*	**H**
A T	**R**
T	**A**
cola vacia	**T**

Figura 18.13. Recorrido en anchura del grafo de Figura 18.12.

El recorrido del grafo en anchura a partir del nodo D del grafo 18.12, ha accedido a todos los nodos del grafo, por ello se puede afirmar que todos los vértices del grafo son alcanzables desde el vértice D.

Grafos Representación de los Grafos RECORRIDO DE UN GRAFO

2. Recorrido en profundidad

La búsqueda de los vértices y aristas de un grafo en profundidad persigue el mismo objetivo que el recorrido en anchura: visitar todos los vértices del grafo alcanzables desde un vértice dado. La diferencia de este recorrido con el recorrido en anchura es el orden en que se procesan los vértices adyacentes; en este nuevo recorrido el orden viene determinado por la estructura pila. El recorrido empieza por un vértice v del grafo; éste se marca como visitado y se mete en la pila. Después se recorre en profundidad cada vértice adyacente a v no visitado; así hasta que no haya más vértices adyacentes no visitados. Esta estrategia de examinar los vértices se denomina en profundidad porque la dirección de "visitar" es hacia adelante mientras que sea posible; al contrario que la búsqueda en anchura que primero visita todos los vértices posibles en amplitud. Como ejemplo, se recorre en profundidad el grafo dirigido de la Figura 18.12. El recorrido se inicia a partir del nodo D, se marca y se mete en la pila. Ahora, iterativamente, se quita el nodo cabeza, se procesa, se meten en la pila sus adyacentes no marcados y se marcan. Los sucesivos elementos de la pila se muestran en la Figura 18.14. En la columna de vértices procesados el nodo en negrita es el que se visita en esa pasada, en la pila los vértices en cursiva son los que se meten en la pila y a la vez son marcados.

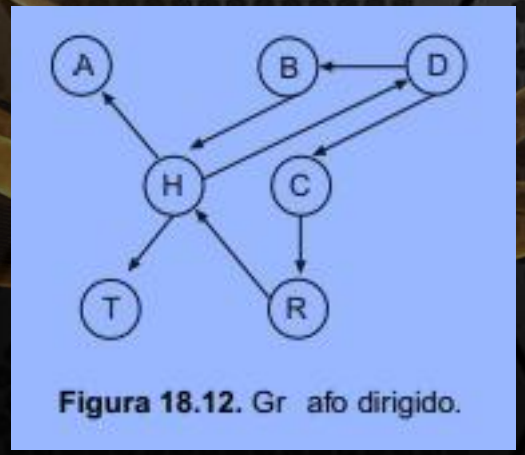


Figura 18.12. Gr afo dirigido.

PILA	Vértices procesados
D	
B *C*	**D**
B *R*	**C**
B *H*	**R**
B A *T*	**H**
B A	**T**
B	**A**
pila vacia	**B**

Figura 18.14. Recorrido en profundidad del grafo de Figura 18.12.

Grafos Representación de los Grafos

Implementación: clase **RecorreGrafo**

Los algoritmos comunes con grafos pueden implementarse como métodos de la clase GrafoAdcia1 o de GrafoMatriz. Sin embargo, se ha preferido agrupar estos métodos en una nueva clase RecorreGrafo. Los algoritmos de recorrido en profundidad y en anchura se encuentran como métodos públicos en la clase RecorreGrafo, recibiendo como argumento el grafo (con matriz o con listas de adyacencia) y el vértice de partida del recorrido y realizando el recorrido del grafo independientemente de la implementación realizada o si la identificación del vértice se realiza con un entero o con un nombre

```
class RecorreGrafo {
  public:
    int CLAVE; // Clave para vértice no marcado
  RecorreGrafo() {
    CLAVE = 0xffff;
  } //Clave vértice no marcado
  //operaciones con los recorridos
  int * recorrerAnchura(GrafoMatriz g, string org);
  int * recorrerAnchura(GrafoAdcia g, string org);
  int * recorrerProf(GrafoMatriz g, string org);
  int * recorrerProf(GrafoAdcia g, string org);
  // sobrecarga de los métodos pasando el número de vértice
  int * recorrerAnchura(GrafoMatriz g, int v);
  //...
};
```

Grafos Representación de los Grafos

1. Implementar Recorrido en anchura

El recorrido en anchura utiliza una cola en la que se guardan los vértices adyacentes al que se ha procesado. Para determinar si un vértice está o no marcado se pueden seguir varias alternativas, una de ellas utiliza el array m[], de tantas posiciones como vértices y con una correspondencia directa entre índice y número de vértice, para indicar si un nodo está marcado. El estado de nodo i no marcado se establece con una clave previamente determinada, y si está marcado tiene un número finito. Al recorrer el grafo se puede obtener cierta información relativa a los vértices. En la codificación que se realiza a continuación, se guarda en m[i] el número mínimo de aristas para cualquier camino desde el vértice de partida hasta el vértice i. El vértice de partida, v, se inicializa m[v] = 0 ya que los caminos parten desde ese vértice; las otras posiciones de m[i] se inicializan al valor clave que expresa vértice no marcado. Cada vez que un vértice w es añadido al recorrido, si (u, w) es la arista que lo incluye entonces se hace m[w] = m[u] + 1 ya que el camino a w tiene una arista más que el que lleva a u. La implementación que se presenta está hecha para un grafo representado por matriz de adyacencia y cuyo vértice origen viene dado por un número entero u. Si el vértice viene identificado por su nombre (string) basta con buscar previamente el número de vértice asociado, tal y como se hace en la implementación de la segunda versión de la función miembro recorreAnchura() sobrecargada. La realización con listas de adyacencia es exactamente la misma que la realizada con matriz de adyacencia ya que los métodos de ambas implementaciones tienen igual nombre y realizan las mismas funciones, por lo que sólo se diferencian en la declaración de los métodos.

Grafos Representación de los Grafos

Implementación: clase RecorreGrafo

El recorrido en anchura necesita de la case Cola que debe ser incluida en la codificación. Los métodos mínimos que debe contener son los siguientes:

```
class Cola {
  protected:
    // atributos
    public:
     void insertar(int v); // añade v a la cola
     bool colaVacia(); // decide si la cola está vacía
     int quitar() {}; // borra y retorna el primer elemento
     // resto de métodos
};
```

La versión para el caso de que la identificación del vértice se realice con el nombre es:

```
int * RecorreGrafo::recorrerAnchura(GrafoMatriz
g, string org) {
  int v = g.numVertice(org);
  if (v < 0) throw " Vértice origen no existe";
  recorrerAnchura(g, v);
}
```

Para visitar todos los vértices del grafo, una vez que ha terminado el recorrido a partir de uno dado, v, hay que buscar si queda algún vértice sin marcar, en cuyo caso se vuelve a recorrer a partir de él y así, sucesivamente, hasta que todos los vértices estén marcados.

```
int * RecorreGrafo::recorrerAnchura(GrafoMatriz g, int v) {
  int w;   Cola cola;   int * m;
  if (v < 0 || v > g.OnumeroDeVertices())
    throw "Vértice no existe";
  m = new int[g.OnumeroDeVertices()];   // inicializa los vértices como no marcados
  for (int i = 0; i < g.OnumeroDeVertices(); i++)
    m[i] = CLAVE;
  m[v] = 0; // vértice origen queda marcado
  cola.insertar(v);
  while (!cola.colaVacia()) {
    int cw;
    cw = cola.quitar();
    w = cw;
    cout << "Vértice " << w << g.Overtice(w).OnomVertice() <<
      " visitado"; // inserta en la cola los adyacentes de w no marcados
    for (int u = 0; u < g.OnumeroDeVertices(); u++)
      if (g.Ovalor(w, u) && m[u] == CLAVE) {
        // se marca vértice u con número de arcos hasta el w
        m[u] = m[w] + 1;
        cola.insertar(u);
      }
  }
  return m;
}
```

Grafos Representación de los Grafos

2. Recorrido en profundidad

La implementación utiliza una pila para establecer el orden de recorrido. También se podría realizar una implementación recursiva para evitar la utilización de la pila. Como en el recorrido en anchura, los vértices ya visitados se marcan en el array m[](pero ahora no representa el camino mínimo de v al vértice, sino el número de aristas del camino que se ha obtenido en el recorrido en profundidad). El grafo está representado mediante listas de adyacencia. Puede observarse que la implementación realizada es similar a la del recorrido en anchura cuando se hizo con el grafo implementado con matriz de adyacencia. Sólo se diferencia en el uso de una pila(propio del recorrido en profundidad), y que el parámetro g es ahora un objeto de la clase GrafoAdcia2, en lugar de serlo de la clase GrafoMatriz. Se incluye, además, la declaración mínima de la clase Pila, que se supone ya implementada y que debe incluirse

```
class Pila {
  protected:
    // atributos
    public:
    void insertar(int v); // añade v a la pila
  bool pilaVacia(); // decide si la pila está vacía
  int quitar() {}; // borra y retorna el primer elemento
  // resto de métodos
};
int * RecorreGrafo::recorrerProf(GrafoAdcia g, int v) {
  int w;
  Pila pila;
  int * m;
  if (v < 0 || v > g.OnumeroDeVertices())
    throw "Vértice no existe";
  m = new int[g.OnumeroDeVertices()];
  // inicializa los vértices como no marcados
  for (int i = 0; i < g.OnumeroDeVertices(); i++)
    m[i] = CLAVE;
  m[v] = 0; // vértice origen queda marcado
  pila.insertar(v);
  while (!pila.pilaVacia()) {
    int cw;
    cw = pila.quitar();
    w = cw;
    cout << "Vértice " << w << g.Overtice(w).OnomVertice() <<
      " visitado";
    // inserta en la pila los adyacentes de w no marcados
    for (int u = 0; u < g.OnumeroDeVertices(); u++)
      if (g.Ovalor(w, u) && m[u] == CLAVE) {
        // se marca vértice u con número de arcos hasta el
        m[u] = m[w] + 1;
        pila.insertar(u);
      }
  }
  return m;
}
int * RecorreGrafo::recorrerProf(GrafoAdcia g, string org) {
  int v = g.numVertice(org);
  if (v < 0) throw " Vértice origen no existe";
  recorrerProf(g, v);
}
```

Grafos| Graph

Conexiones de un Grafo

Grafos Representación de los Grafos

CONEXIONES EN UN GRAFO

Al modelar un conjunto de objetos y sus relaciones mediante un grafo, una de las cuestiones, que generalmente interesa saber es si desde cualquier vértice se puede acceder al resto de los vértices del grafo, es decir, si todos los vértices están conectados, o simplemente, si el grafo es conexo. Para un grafo dirigido, la conectividad entre todos los vértices se denomina: grafo fuertemente conexo.

Componentes conexas de un grafo

Un grafo no dirigido G es conexo si existe un camino entre cualquier par de vértices que forman el grafo. En el caso de que el grafo no sea conexo resulta interesante determinar aquellos subconjuntos de vértices que mutuamente están conectados, esto es, las componentes conexas del mismo. El algoritmo para determinar las componentes conexas de un grafo G se basa en el recorrido del grafo (en anchura o en profundidad).
Los pasos a seguir son:

1. Realizar un recorrido del grafo a partir de cualquier vértice w.
2. Si en el recorrido se han marcado todos los vértices, entonces el grafo es conexo.
3. Si el grafo no es conexo, los vértices marcados forman una componente conexa.
4. Se toma un vértice no marcado, z, y se realiza de nuevo el recorrido del grafo a partir de z. Los nuevos vértices marcados forman otra componente conexa.
5. El algoritmo termina cuando todos los vértices han sido marcados (visitados).

Grafos Representación de los Grafos

Componentes fuertemente conexas de un grafo

Un grafo dirigido fuertemente conexo es aquél en el cual existe un camino entre cualquier par de vértices del grafo. De no ser fuertemente conexo se pueden determinar componentes fuertemente conexas del grafo. La Figura 18.15 muestra un grafo dirigido y sus dos componentes fuertemente conexas.

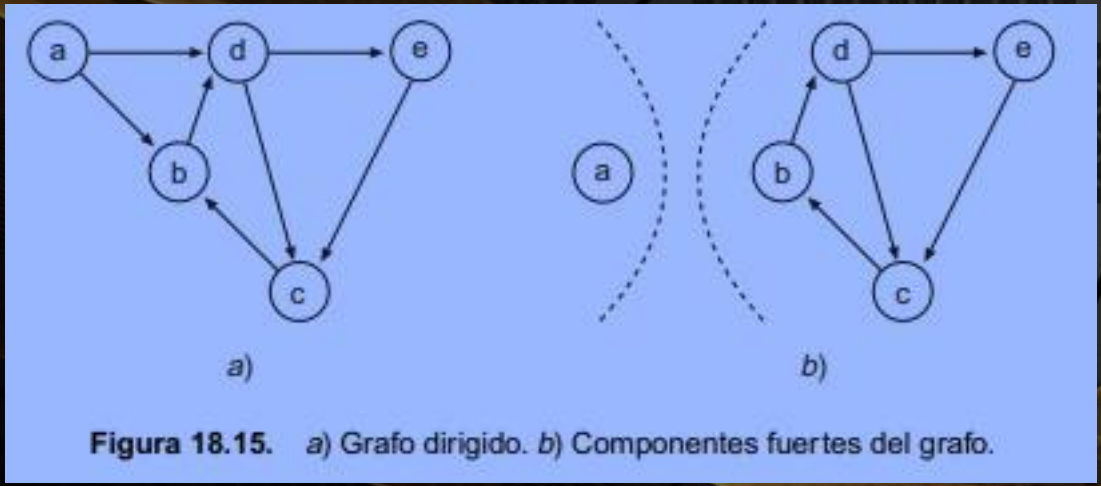


Figura 18.15. *a*) Grafo dirigido. *b*) Componentes fuertes del grafo.

El recorrido en profundidad o en anchura de un grafo a partir de un vértice dado permite diseñar un algoritmo para encontrar si un grafo es fuertemente conexo, o en su caso determinar las componentes fuertemente conexas.

A continuación se indican los pasos que sigue:

1. Obtener el conjunto de vértices descendientes del vértice de partida v, D(v), incluido el propio vértice v.
2. Obtener el conjunto de ascendientes de v, A(v), incluido el propio vértice v.
3. Los vértices comunes del conjunto de descendientes y ascendientes de v: D(v) ➤ A(v), es el conjunto de vértices que forman la componente fuertemente conexa a la que pertenece v. Si este conjunto es el conjunto de todos los vértices del grafo, entonces es fuertemente conexo.
4. Si no es un grafo fuertemente conexo se selecciona un vértice cualquiera, w, que no esté en ninguna componente fuerte de las encontradas (w ← D(v) ➤ A(v)) y se procede de la misma manera, es decir, se repite a partir del paso1 hasta obtener todas las componentes fuertes del grafo.

Grafos Representación de los Grafos

Componentes fuertemente conexas de un grafo

Para buscar los vértices descendientes de v se realiza un recorrido en profundidad o en anchura del grafo a partir de v. Los vértices que son alcanzados se guardan en el conjunto D. Los vértices ascendientes de v son aquellos desde los que se puede alcanzar a v. Por consiguiente, se obtiene el grafo inverso, cambiando el sentido de los arcos, a continuación se recorre en profundidad o en anchura el grafo inverso a partir de v. Los vértices alcanzados en el recorrido son los ascendientes de v. Al recorrer el grafo de la Figura 18.15 en profundidad, a partir del vértice d, el conjunto de vértices que alcanza: {d, c, b, e}. Repitiendo el recorrido en el grafo inverso, Figura 18.16, se obtiene los vértices ascendientes: {d, b, c, e}. Los vértices comunes: (d, b, c, e), forman una componente fuertemente conexa

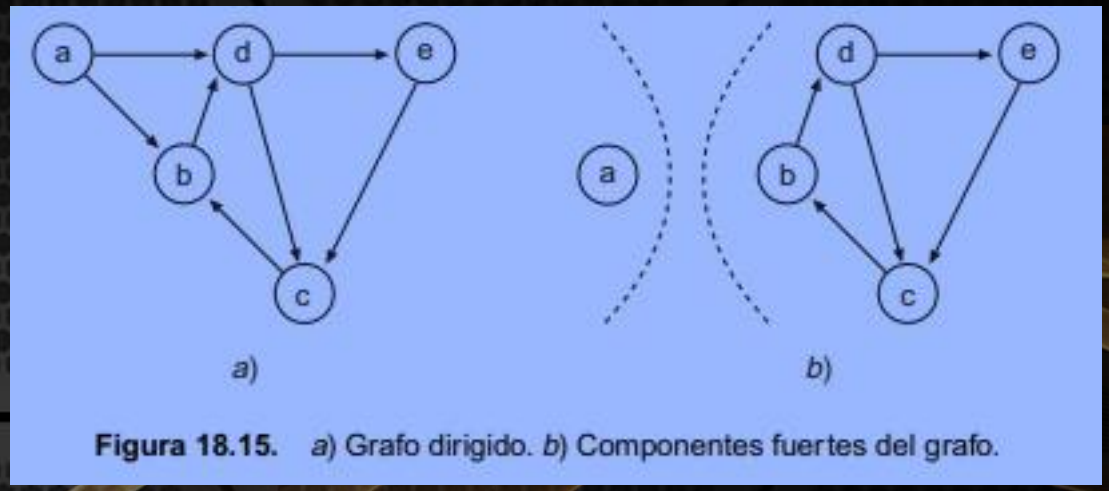


Figura 18.15. *a*) Grafo dirigido. *b*) Componentes fuertes del grafo.

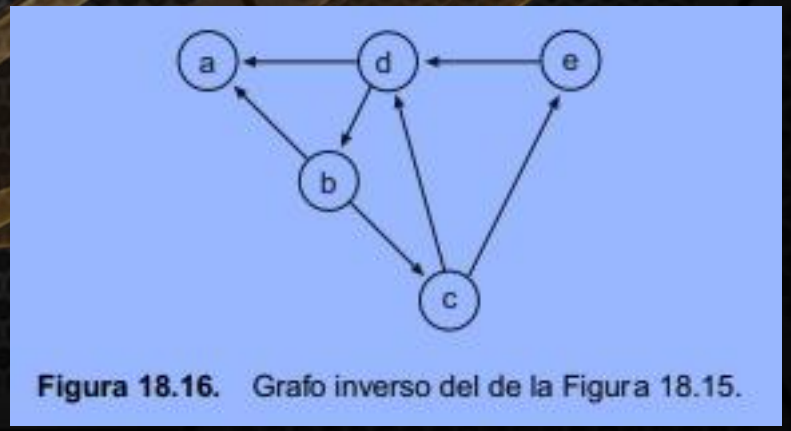
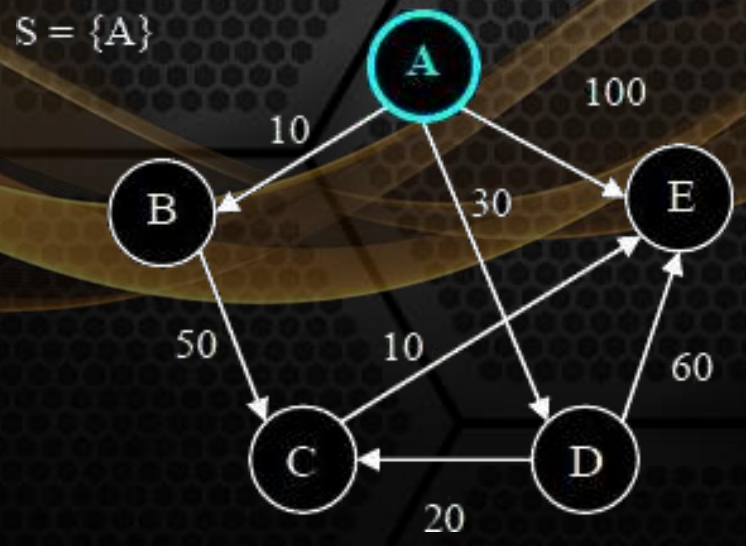


Figura 18.16. Grafo inverso del de la Figura 18.15.

Grafos | Graph

Matriz de Caminos: cierre transitivo



Matriz de Costos : C

	A	B	C	D	E
A	∞	10	∞	30	100
B	∞	∞	50	∞	∞
C	∞	∞	∞	∞	10
D	∞	∞	20	∞	60
E	∞	∞	∞	∞	∞

Grafos Representación de los Grafos

MATRIZ DE CAMINOS. CIERRE TRANSITIVO

Dado un grafo G, un camino de longitud n desde el vértice v0 hasta el vértice vn es una secuencia de n+1 vértices v0, v1, v2,..., vn tal que el vértice inicial es v0, el vértice final vn y son adyacentes. Encontrar caminos entre un par de vértices, de una determinada longitud es una tarea relativamente sencilla, aunque poco eficiente, si se tiene la matriz de adyacencia del grafo Si se considera que la matriz de adyacencia, A, es de tipo bool, la expresión Ai,k && Ak,j es verdadera si y sólo si los valores de ambos operandos lo son. Esta hipótesis implica que hay un arco desde el vértice i al vértice k y otro desde el vértice k al j. También se puede afirmar, que si Ai,k && Ak,j es verdadera existe un camino de longitud 2 desde el vértice i al j. Ese sencillo razonamiento se puede ampliar a la siguiente expresión:

$$(A_{i1} \;\&\&\; A_{1j}) \;||\; (A_{i2} \;\&\&\; A_{2j}) \;||\; \ldots \;||\; (A_{in} \;\&\&\; A_{nj})$$

Que la expresión sea verdadera implica que hay, al menos, un camino de longitud 2 desde el vértice i al vértice j que pase por el vértice 1, o por el 2 ... o por el vértice n. Si ahora se cambia el operador && por el producto y || por suma, la expresión es el elemento Aij de la matriz producto A2 . La conclusión es inmediata: los elementos (Aij) de la matriz A2 son distintos de cero si existe un camino de longitud 2 desde el vértice i al vértice j, i,j = 1..n. El razonamiento se puede extender para encontrar caminos de longitud 3; realizando el producto matricial A2xA = A3 se encuentran caminos de longitud 3 entre cualquier par de vértices del grafo.

Grafos Representación de los Grafos

A tener en cuenta

Una manera de encontrar los caminos de longitud m entre cualquier par de vértices de un grafo es mediante el producto matricial de A^{m-1} por la matriz de adyacencia A.

El elemento c_{ij} de la matriz producto se obtiene **multiplicando** cada elemento de la **fila i** de la matriz A por cada elemento de la **columna j** de la matriz B y **sumándolos**.

$$A \cdot B = \begin{pmatrix} 2 & 0 & 1 \\ 3 & 0 & 0 \\ 5 & 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 0 \end{pmatrix} =$$

$$= \begin{pmatrix} 2\cdot1+0\cdot1+1\cdot1 & 2\cdot0+0\cdot2+1\cdot1 & 2\cdot1+0\cdot1+1\cdot0 \\ 3\cdot1+0\cdot1+0\cdot1 & 3\cdot0+0\cdot2+0\cdot1 & 3\cdot1+0\cdot1+0\cdot0 \\ 5\cdot1+1\cdot1+1\cdot1 & 5\cdot0+1\cdot2+1\cdot1 & 5\cdot1+1\cdot1+1\cdot0 \end{pmatrix} =$$

$$= \begin{pmatrix} 3 & 1 & 2 \\ 3 & 0 & 3 \\ 7 & 3 & 6 \end{pmatrix}$$

A continuación se siguen los mismos razonamientos para obtener caminos del grafo de la Figura 18.17, cuya matriz de adyacencia es la siguiente:

$$A = \begin{vmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{vmatrix}$$

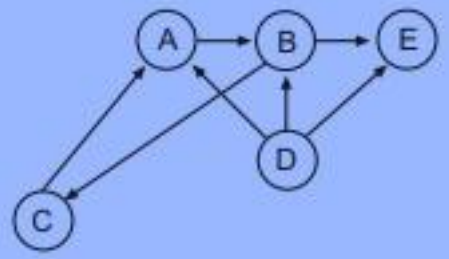


Figura 18.17. Grafo dirigido de 5 vértices.

El producto matricial AxA, considerando todo valor distinto de 0 como 1 (verdadero):

$$A^2 = \begin{vmatrix} 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{vmatrix}$$

Que, por ejemplo, A15 tenga el valor 1 significa que hay un camino de longitud 2, desde A – E, formado por los arcos: (A l B l E)

$$A^3 = \begin{vmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{vmatrix}$$

Si se observa A41, su valor es 1 ya que hay un camino longitud 3 desde D – A, formado por los arcos: (D l B l C l A). Realizando el producto matricial, el valor almacenado en cualquier posición, Aij, no sólo indica si hay camino de longitud m, sino, además, el número de caminos de dicha longitud entre los vértices i y j.

Grafos Representación de los Grafos

A recordar

La forma de obtener el número de caminos de longitud k entre cualquier par de vétices de un grafo es obtener el producto matricial A^2, A^3 ... A^k. Entonces el elemento $A_k(i,j)$ contiene el número de caminos de longitud k desde el vértice i hasta el vértice j.

A tener en cuenta

La eficiencia del algoritmo para encontrar el número de caminos de longitud k es muy pobre . El producto matricial se realiza con tres bucles anidados, complejidad cúbica $O(n^3)$, además, este producto se realiza k-1 veces.

Matriz de caminos y cierre transitivo

Sea G un grafo con n vértices, la matriz de caminos de G es la matriz P de n*n elementos, definida

Se encuentra la siguiente relación entre la matriz de caminos P, la matriz de adyacencia A y las sucesivas potencias de A para encontrar los caminos de longitud m: dada la matriz Bn = A + A2 + A3 +...+ An , la matriz de caminos P es tal que un elemento Pij = 1 si y sólo si Bn(i,j) >= 1 y en otro caso Pij = 0. El cierre transitivo o contorno transitivo de un grafo G es otro grafo, G', con los mismos vértices y cuya matriz de adyacencia es la matriz de caminos del grafo G.

$$P_{ij} = \begin{cases} 1 & \textit{si hay un camino desde } v_i \text{ a } v_j \\ 0 & \textit{si no hay camino desde } v_i \text{ a } v_j \end{cases}$$

Grafos | Graph

Ordenación Topológica

Grafos Representación de los Grafos

ORDENACIÓN TOPOLÓGICA

Una de las aplicaciones de los grafos es modelar las relaciones que existen entre las diferentes tareas, hitos, que deben de terminarse para finalizar un proyecto. Entre las tareas hay relaciones de precedencia, una tarea r precede a la tarea t si es necesario que se complete r para poder empezar t. Estas relaciones de precedencia se representan mediante un grafo dirigido en el que los vértices son las tareas o hitos y hay una arista del vértice r al t si el inicio de la tarea t depende de la terminación de r.

Una vez se dispone del grafo interesa obtener una planificación de las tareas para poder realizarlas que constituyen el proyecto, en definitiva, encontrar la ordenación topológica de los vértices que forman el grafo. El grafo que representa estas relaciones de precedencia es un grafo dirigido acíclico, de tal forma que si hay un camino de u a v, entonces, en la ordenación topológica v es posterior a u. Es inmediato que el grafo no puede tener ciclos cuando representa relaciones de precedencia, de haber un ciclo significaría que si r y t son vértices del ciclo, r depende de t y a su vez t depende de la terminación de r.

A tener en cuenta

La ordenación topológica se aplica sobre grafos dirigidos sin ciclos. Es una ordenación lineal, tal que si v es anterior a w entonces hay un camino de v a w. La ordenación topológica no se puede realizar en grafos con ciclos.

Grafos Representación de los Grafos ORDENACIÓN TOPOLÓGICA

La Figura 18.18 muestra un grafo acíclico que modela la estructura de prerequisitos de 8 cursos. Una arista cualquiera (r, s) significa que el curso r debe de completarse antes de empezar el curso s. Por ejemplo en la Figura 18.18, el curso M21 se puede empezar sólo cuando terminen los cursos E11 y T12; en ese sentido E11 y T12 son prerequisitos de M21. Una ordenación topológica de estos cursos es cualquier secuencia de cursos que cumple los requerimientos (prerequisitos). Entonces, para un grafo dirigido acíclico no tiene por qué existir una única ordenación topológica. Del grafo de requisitos de la Figura 18.18 se obtienen estas ordenaciones topológicas: E11 - T12 - M12 - C22 - R23 - S31 - S32 - T41 T12 - E11 - R23 - C22 - M21 - S31 - S32 - T41 Un grafo G dirigido y sin ciclos se denomina un gda o grafo acíclico. Los gda,s son útiles para la representación de ordenaciones parciales. Una ordenación parcial R en un conjunto C es una relación binaria de precedencia tal que: 1. Para todo u ↓ C, u no está relacionado con sí mismo, uRu es falso, por consiguiente, la relación R es no reflesiva. 2. Para todo u, v, w ↓ C, si uRv y vRw entonces uRw. La relación R es transitiva. Tal relación R sobre C se llama ordenación parcial de C. La relación de inclusión en conjuntos es un ejemplo de ordenación parcial. Un grafo G sin ciclos se puede considerar un conjunto parcialmente ordenado

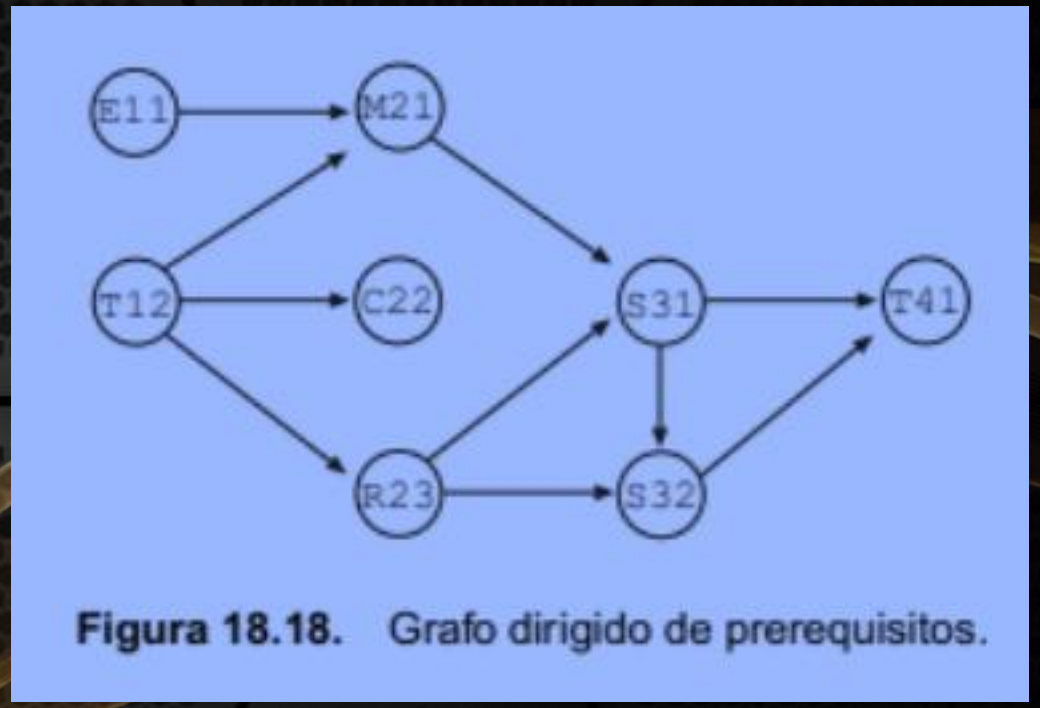


Figura 18.18. Grafo dirigido de prerequisitos.

Nota

Se puede probar que un g rafo es *acíclico* realizando un recorr ido en profundidad, de tal forma que si se encuentr a un arco de *retroceso* en el árbol de búsqueda, el g rafo tiene al menos un ciclo, y si no se encuentr a es acíclico

Grafos Representación de los Grafos

ORDENACIÓN TOPOLÓGICA

Algoritmo para obtener una ordenación topológica

El algoritmo, en primer lugar, busca un vértice (una tarea) sin predecesores o prerequisitos, esto es, que no tenga arcos de entrada (este vértice tiene que existir ya que si no lo hubiera no se podría comenzar por ninguna tarea). El v, pasa a formar parte de la ordenación T. A continuación, todos los arcos que salen de v son eliminados, debido a que el prerequisito v ya se ha satisfecho. La estrategia se repite, se toma otro vértice w sin arcos incidentes (que llegan a él), se incorpora a la ordenación T y se elimina todos los arcos que salen de el. Se itera este proceso hasta completar la ordenación topológica. Que un vértice v no tenga arcos incidentes se expresa con el grado de entrada, gradoEntrada(v)== 0. Eliminar los arcos que salen de v, implica que gradoEntrada(w) de todos los vértices w adyacentes de v disminuyen en 1. Por consiguiente, el algoritmo comienza calculando el grado de entrada de cada vértice del grafo. Los vértices cuyo grado entrada es 0 se guardan en una Cola. El elemento frente de la cola, v, pasa a formar parte de la ordenación T. A continuación, se disminuye en 1 el grado de entrada de los adyacentes de v, y aquellos vértices cuyo grado de entrada han pasado a ser 0 se meten en la cola. Este proceso se itera hasta que la cola esté vacía

A tener en cuenta

Si al aplicar el algor itmo de ordenación topológica ha y vér tices del g rafo que aún no han pasado a formar parte de la ordenación y la cola está v acia, entonces el grafo tiene ciclos.

Grafos Representación de los Grafos

Implementación del algoritmo de ordenación topológica

La codificación del algoritmo depende de la representación del grafo, con matriz de adyacencia o listas de adyacencia. Si el grafo tiene relativamente pocos arcos, es poco denso, la matriz de adyacencia tiene muchos ceros, es una matriz esparcida y, por consiguiente, el grafo se representa con listas de adyacencia. Para grafos dirigidos densos se prefiere, por eficiencia, la matriz de adyacencia.

Con independencia de la representación, se utiliza una Cola, para almacenar los vértices con grado de entrada 0. La codificación que a continuación se presenta supone el grafo representado con listas de adyacencia (clase GrafoAdcia); los vértices de la ordenación topológica se escribe por pantalla y se almacenan en el vector T[]. El número de vértices almacenados en el vector se retorna en el parámetro por referencia j. También se escribe el método gradoEntrada()que calcula el grado de entrada de un vértice de un grafo que recibe como parámetro.

A tener en cuenta

La complejidad del algoritmo *ordenación topológica* de un grafo, representado con listas de adyacencia, es $O(a+n)$ siendo a el número de arcos y n el de vér tices. Si la representación del grafo es con una matriz de adyacencia, la complejidad es $O(n^2)$.

```
int gradoEntrada(GrafoAdcia g, int v) {
  int cuenta = 0;
  for (int origen = 0; origen < g.OnumeroDeVertices(); origen++) {
    if (g.adyacente(origen, v)) // arco incidente a v
      cuenta++;
  }
  return cuenta;
}
// Método para obtener una ordenación topológica.  Muestra los vértices que pasan a formar parte de la
// ordenación, y se almacenan T[]. El número de vértices lo da en j
void ordenTopologica(GrafoAdcia g, int T[], int & j) {
  int * arcosInciden;   int v, w;
  Cola cola;
  j = 0;
  arcosInciden = new int[g.OnumeroDeVertices()];
  // grado de entrada de cada vértice
  for (v = 0; v < g.OnumeroDeVertices(); v++)
    arcosInciden[v] = gradoEntrada(g, v);
  cout << "\n Ordenación topológica ";
  for (v = 0; v < g.OnumeroDeVertices(); v++)
    if (arcosInciden[v] == 0)
      cola.insertar(v); // vértices con grado de entrada 0
  while (!cola.colaVacia()) {
    w = cola.quitar();
    cout << " " << w;
    T[j++] = w;
    Lista itl = g.listaAdyc(w); // lista de adyacencia de g
    Nodo * l = itl.Ol(); // puntero al primer nodo de lista iterado de lista
    // decrementa grado entrada de vértices adyacentes
    while (l != NULL) // mientras queden nodos en la lista
    {
      v = l -> Oarco().Odestino(); //vértice destino del arco del nodo
      arcosInciden[v]--;
      if (arcosInciden[v] == 0)
        cola.insertar(v); // vértice con grado de entrada 0
      l = l -> Osig();
    }
  }
}
```

Grafos| Graph

MATRIZ DE CAMINOS:

ALGORITMO DE WARSHALL

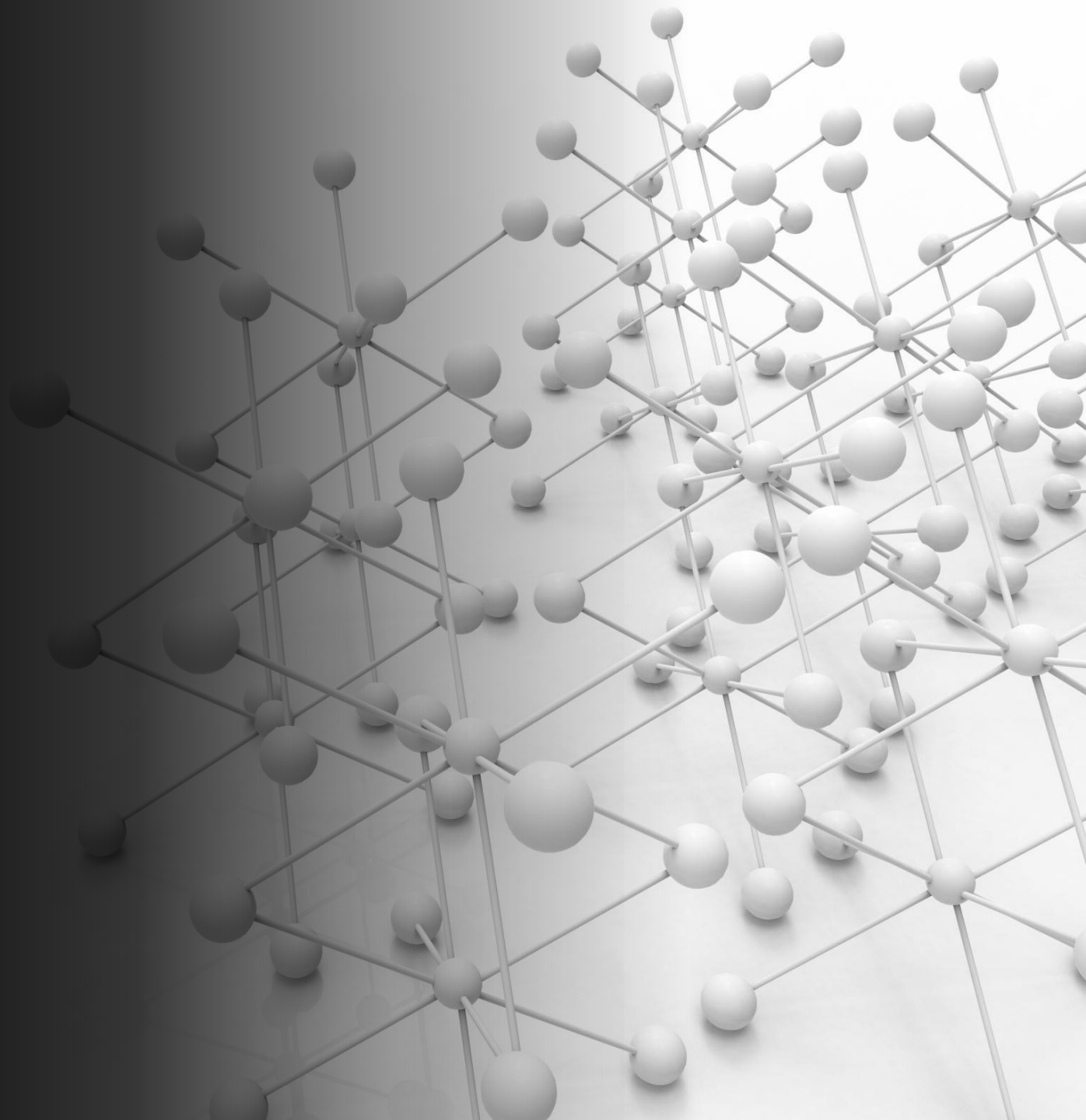
ALGORITMO DE DIJKSTRA

Grafos | Graph

MATRIZ DE CAMINOS: ALGORITMO DE WARSHALL

Grafos Representación de los Grafos

- MATRIZ DE CAMINOS: ALGORITMO DE WARSHALL



Grafos Representación de los Grafos

MATRIZ DE CAMINOS: ALGORITMO DE WARSHALL

Este algoritmo calcula la matriz de caminos P (también llamado cierre transitivo) de un grafo G de n vértices, representado por su matriz de adyacencia A. La estrategia que sigue el algoritmo consiste en definir, a nivel lógico, una secuencia de matrices n-cuadradas P0, P1, P2, P3 ... Pn. Los elementos de cada una de las matrices Pk[i,j] tienen el valor 0 si no hay camino, 1 si hay camino del vértice i y al j que no pasa por otro vértice a no ser que estén comprendidos entre 1 ... k La matriz P0 es la matriz de adyacencia. Los elementos de P1 son igual a 1 si lo son los de P0, y los que son 0 en P0 pasan a 1 en P1 si se puede formar un camino entre el par de vértices, con la ayuda del vértice 1. En definitiva, P1[i,j] = 1 si lo es P0[i,j], o bien, hay un camino:



En cada paso se incorpora un nuevo vértice, el que coincide con el índice de la matriz P, a los anteriores para poder formar camino.

$$P_k[i,j] = \begin{cases} 1 & \text{si hay un camino simple de } v_i \text{ a } v_j \text{ que no pasa por otro vértice a no ser por los que están comprendidos entre los vértices } 1 \ldots k. \\ 0 & \text{en otro caso.} \end{cases}$$

$$P_n[i,j] = \begin{cases} 1 & \text{si hay un camino simple de } v_i \text{ a } v_j \text{ que no pasa por ningún otro vértice a no ser por los que están comprendidos entre los vértices } 1 \ldots n. \\ 0 & \text{en otro caso.} \end{cases}$$

$$P_0[i,j] = \begin{cases} 1 & \text{si hay un arco del vértice } i \text{ al } j \\ 0 & \text{en otro caso.} \end{cases}$$

$$P_1[i,j] = \begin{cases} 1 & \text{si hay un camino simple de } v_i \text{ a } v_j \text{ que no pasa por ningún otro vértice a no ser por el vértice } 1. \\ 0 & \text{en otro caso.} \end{cases}$$

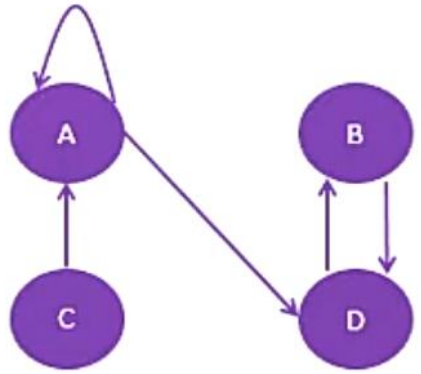
$$P_2[i,j] = \begin{cases} 1 & \text{si hay un camino simple de } v_i \text{ a } v_j \text{ que no pasa por otro vértice a no ser por los que están comprendidos entre los vértices } 1 \ldots 2. \\ 0 & \text{en otro caso.} \end{cases}$$

$$P_3[i,j] = \begin{cases} 1 & \text{si hay un camino simple de vi a vj que no pasa por otro vértice a no ser por los que están comprendidos entre los vértices } 1 \ldots 3. \\ 0 & \text{en otro caso.} \end{cases}$$

Grafos Representación de los Grafos

- MATRIZ DE CAMINOS: ALGORITMO DE WARSHALL

ALGORITMO DE WARSHALL



W0

	A	B	C	D
A	1	0	0	1
B	0	0	0	1
C	1	0	0	0
D	0	1	0	0

Matriz de cierre transitivo.
Los caminos posibles No directos

W1

	A	B	C	D
A	1	0	0	1
B	0	0	0	1
C	1	0	0	1
D	0	1	0	0

W2

	A	B	C	D
A	1	0	0	1
B	0	0	0	1
C	1	0	0	1
D	0	1	0	1

W3

	A	B	C	D
A	1	0	0	1
B	0	0	0	1
C	1	0	0	1
D	0	1	0	1

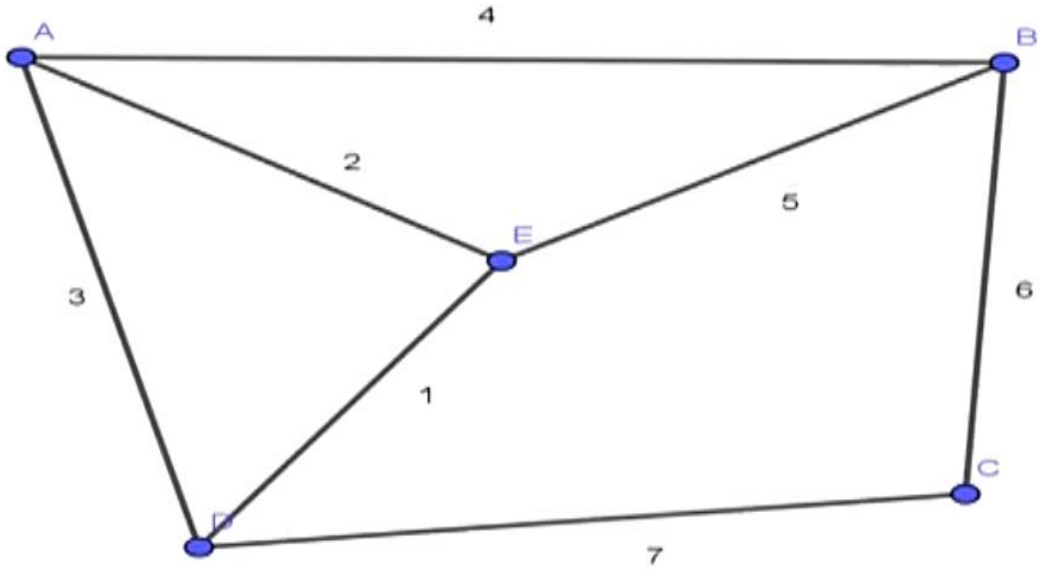
W4

	A	B	C	D
A	1	1	0	1
B	0	1	0	1
C	1	1	0	1
D	0	1	0	1

1. Se creas la matrix de Adyacencia
2. Se copia A
3. Se busca donde hay 1 coincidencia.
4. Relleno con lo anterior los Faltante
5. Itera para b c y d

Grafos Representación de los Grafos

- MATRIZ DE CAMINOS: ALGORITMO DE WARSHALL



Matrices requeridas

Matriz de distancias					
	A	B	C	D	E
A	0	4	X	3	2
B	4	0	6	X	5
C	X	6	0	7	X
D	3	X	7	0	1
E	2	5	X	1	0

Matriz de recorrido					
	A	B	C	D	E
A	0	B	C	D	E
B	A	0	C	D	E
C	A	B	0	D	E
D	A	B	C	0	E
E	A	B	C	D	0

1. Creo la matriz de distancias
2. Creo la matriz de recorrido
3. Me paro en A-A sumo los valores fila-columna, si es menor reemplazo
4. Anoto en la celda de Matriz de Recorrido la letra con la que estoy.
5. Itero hasta que no haya mas “Infinito”

Grafos Representación de los Grafos

Implementación del algoritmo de Warshall

Se codifica el algoritmo para un grafo G representado por su matriz de adyacencia. Él método devuelve la matriz de caminos P reservando previamente espacio en memoria. Se inicializa P con la matriz de adyacencia, y se aplica el algoritmo de programación dinámica descrito por Warshall.

```
int ** matrizCaminos(GrafoMatriz g) {
  int n = g.OnumeroDeVertices();
  typedef int * pint; // pint es tipo que apunta a punteros enteros
  int ** P = new pint[n]; //P apunta a un array de punteros enteros
  for (int i = 0; i < n; i++)
    P[i] = new int[n]; // P es ahora una matriz de enteros
  // Se dispone de memoria para la matriz de caminos
  // Se obtiene la matriz inicial: matriz de adyacencia
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      P[i][j] = g.adyacente(i, j) ? 1 : 0;
  // se obtienen, virtualmente, a partir de P0, las sucesivas
  // matrices P1, P2, P3 ,..., Pn-1, Pn que es la matriz de caminos
  for (int k = 0; k < n; k++) // etapas de programación dinámica
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (P[i][k] * P[k][j])
          //observe el índice k de la programación dinámica
          P[i][j] = 1;
  return P;
}
```

Norma sobre eficiencia

La complejidad del algoritmo de *Warshall* es cúbica, $O(n^3)$ siendo n el número de vértices. Esto hace que el tiempo de ejecución crezca rápidamente par a grafos de relativamente muchos nodos.

Grafos | Graph

CAMINOS MÍNIMOS CON UN SOLO ORIGEN: ALGORITMO DE DIJKSTRA

Grafos Representación de los Grafos

CAMINOS MÍNIMOS CON UN SOLO ORIGEN: ALGORITMO DE DIJKSTRA

Uno de los problemas que se plantean con relativa frecuencia es el de determinar la longitud del camino más corto entre un par de vértices de un grafo. Por ejemplo, determinar la mejor ruta (menor tiempo) para ir desde un lugar a un conjunto de centros de la ciudad. Para este tipo de problemas se considera un grafo dirigido y valorado , es decir, cada arco (vi, vj)del grafo tiene asociado un coste cij. La longitud del camino más corto es la suma de los costes de los arcos que forman el camino. Matemáticamente, si v1,v2 …vk es la secuencia de vértices que forman un camino:

$$\text{Longitud de camino} = \sum_{i=1}^{k-1} C_{i,i+1}$$

se pretende encontrar el camino de longitud o coste mínimo. Otro problema relativo a los caminos entre dos vértices v1, vk, es encontrar aquél de menor número de arcos para ir de v1 a vk. En definitiva, encontrar el camino de longitud mínima en un grafo no valorado. Este problema se resuelve con una búsqueda en anchura a partir del Vértice de partida .

El algoritmo que se describe a continuación encuentra el camino mínimo desde un vértice origen al resto de vértices, en un grafo valorado positivamente (con factor de peso positivo).

Este algoritmo recibe el nombre de algoritmo de **Dijkstra** en honor del científico que lo expuso.

Grafos Representación de los Grafos

Algoritmo de Dijkstra

Dado un grafo dirigido G = (V, A) valorado y con factores de peso no negativos, el algoritmo de Dijkstra determina el camino más corto desde un vértice al resto de los vértices del grafo. Éste es un ejemplo típico de algoritmo ávido(voraz) que selecciona en cada paso la solución más optima en aras de resolver el problema. En cada iteración se come el mejor pedazo. Se recuerda que en un grafo valorado, cada arco (vi, vj) tiene asociado un coste cij. De tal forma que si v0 es el vértice origen, y v0, v1, ..., vk es la secuencia de vértices que forman el camino de v0 a vk, entonces $\sum_{i=1}^{k-1} C_{i,i+1}$ es la longitud del camino.

Para reconstruir el camino de coste mínimo que lleva de s a cada vértice v del grafo se almacena, para cada uno de los vértices, el último vértice que hizo el coste mínimo. Entonces, asociado a cada vértice resulta de interés dos datos, el coste mínimo y el último vértice del camino con el que se minimizó el coste. El ejemplo 18.1 hace un seguimiento de los sucesivos pasos que sigue este algoritmo para encontrar los caminos mínimos desde un origen al resto de vértices del grafo

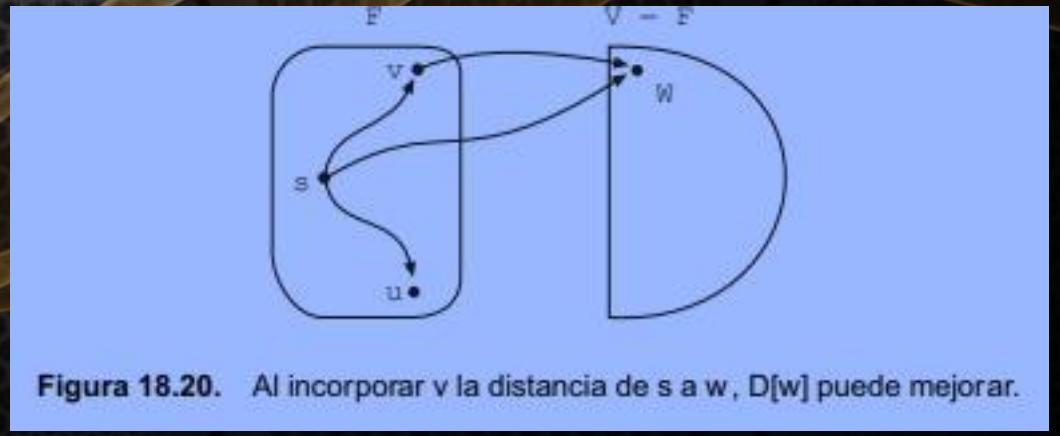
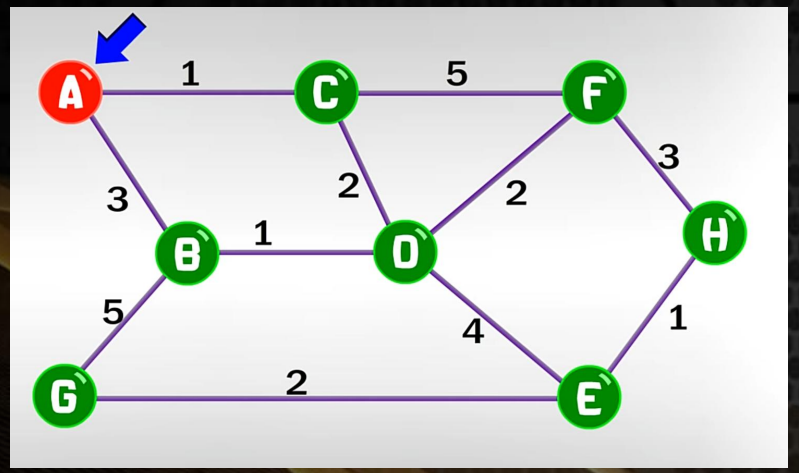
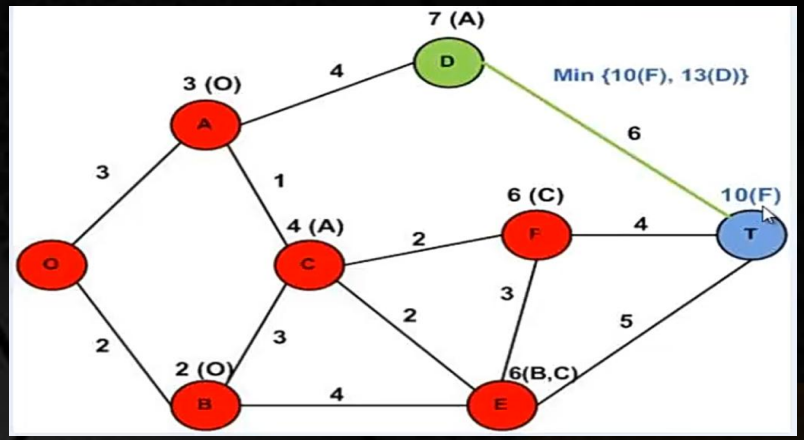


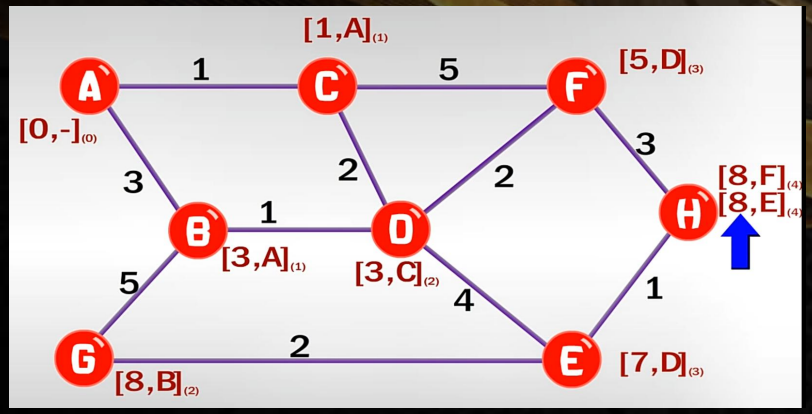
Figura 18.20. Al incorporar v la distancia de s a w, D[w] puede mejorar.

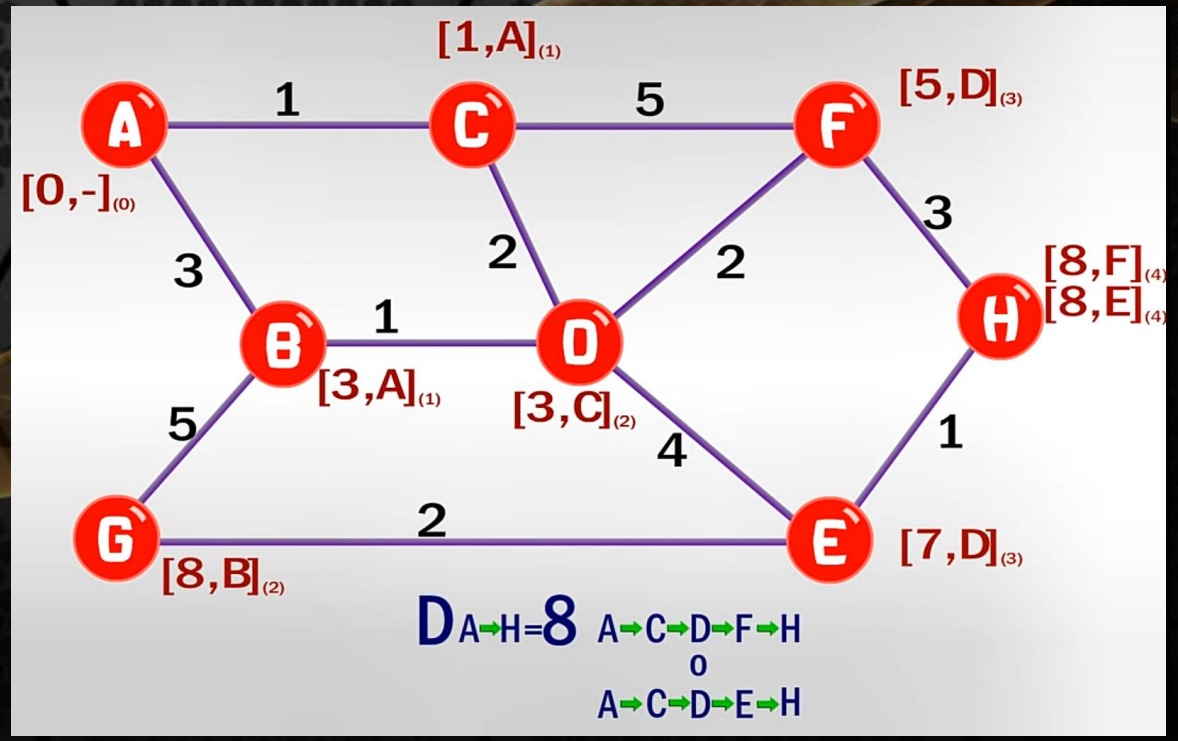
Grafos Representación de los Grafos

Algoritmo de Dijkstra









Grafos | Graph

ÁRBOL DE EXPANSIÓN DE COSTE MÍNIMO

Graph 10

Grafos Representación de los Grafos

ÁRBOL DE EXPANSIÓN DE COSTE MÍNIMO

Los grafos no dirigidos se emplean para modelar relaciones simétricas entre, los vértices del grafo. Cualquier arista (v,w) de un grafo no dirigido es igual que (w,v), se dice que estas aristas están formadas por un par no ordenado de vértices. Como consecuencia directa, la representación de un grafo no dirigido da lugar a matrices simétricas. Una propiedad, que normalmente interesa conocer, de un grafo no dirigido es si para todo par de vértices hay un camino que los une, en definitiva, si el grafo es conexo. A los grafos conexos también se les denomina Red Conectada.

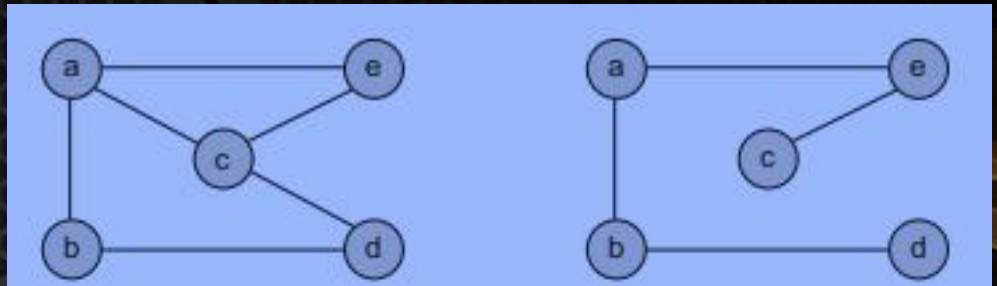


Figura 18.23. Grafo no dirigido y su árbol de expansión.

El problema del árbol de expansión de coste mínimo consiste en buscar un árbol que cubra todos los vértices del grafo, con suma de pesos de aristas mínimo. Los árboles de expansión se aplican en el diseño de redes de comunicación. Un árbol, en una red, es un subconjunto G' del grafo G que es conectado y sin ciclos. Los árboles tienen dos propiedades importantes:

1. Todo árbol de n vértices contiene exactamente n-1 aristas.
2. Si se añade una arista a un árbol entonces resulta un ciclo

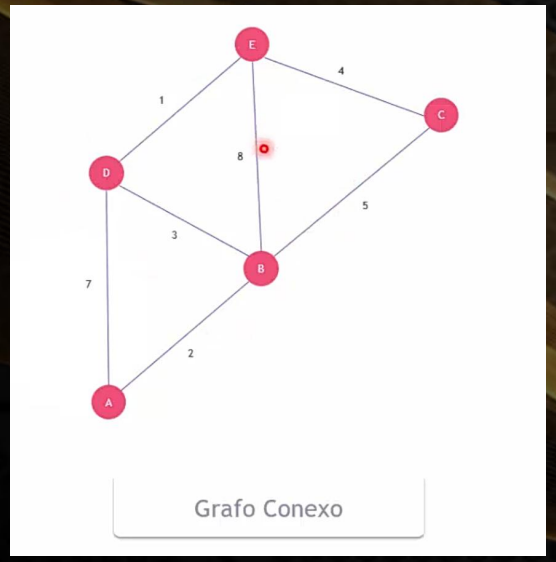
Definición

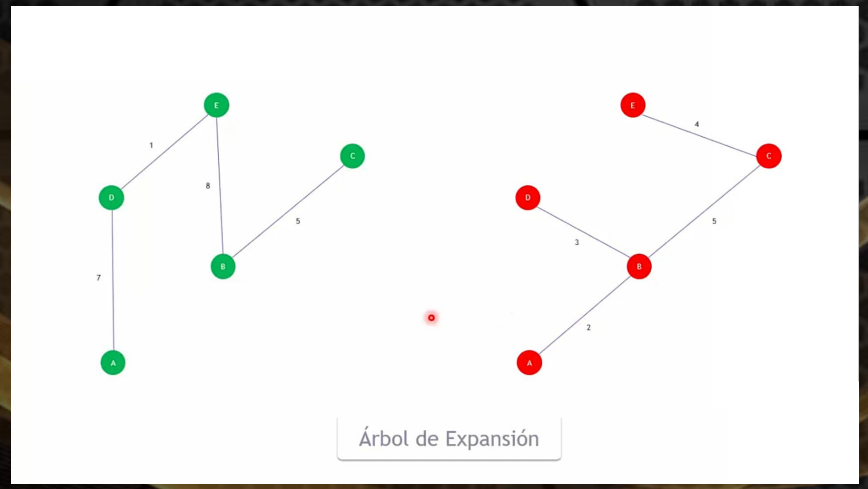
Árbol de expansión de coste mínimo: es un subconjunto del g rafo que abarca a todos los vértices que están conectados, cuyas aristas tienen una suma de pesos mínima.

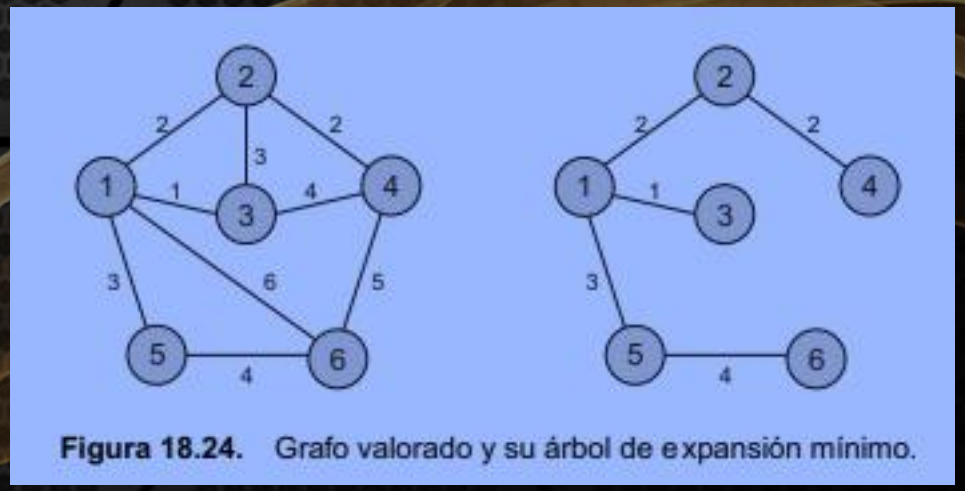
Grafos Representación de los Grafos

ÁRBOL DE EXPANSIÓN DE COSTE MÍNIMO

Buscar un árbol de expansión de un grafo, en una red, es también una forma de averiguar si está conectado. Todos los vértices del grafo tienen que estar en el árbol de expansión para que sea grafo conectado. La Figura 18.23 muestra un grafo no dirigido y su árbol de expansión, el grafo es una red conectada.







El planteamiento del problema es el siguiente: dado un grafo no dirigido ponderado y conexo, encontrar el subconjunto del grafo compuesto por todos los vértices, con conexión entre cada par de vértices, sin ciclos y que cumpla que la suma de los pesos de las aristas sea mínimo. Simplemente, encontrar el árbol de expansión de coste mínimo

Algoritmo de Prim

Algoritmo de Kruscal

Grafos | Graph

ÁRBOL DE EXPANSIÓN DE COSTE MÍNIMO

11

Algoritmo de Prim

Grafos Representación de los Grafos

Algoritmo de Prim

El algoritmo de Prim encuentra el árbol de expansión de coste mínimo (se conectan todos los vértices del grafo con suma de aristas mínima) de un grafo no dirigido. Consta de sucesivos pasos, siguiendo la metodología clásica de los algoritmos voraces: realizar en cada paso “lo mejor que se pueda hacer”. En este problema, lo mejor consiste en incorporar al árbol una nueva arista del grafo de menor longitud. Se parte de un grafo G = (V, A) no dirigido conectado, una red, siendo c(i,j)el peso o coste asociado al arco (vi, vj). Para describir el algoritmo, se supone los vértices están numerados de 1 a n. El conjunto W va a contener los vértices que ya han pasado a formar parte del árbol de expansión. El algoritmo arranca asignando un vértice inicial al conjunto W, por ejemplo el vértice 1: W = {1}. A partir del vértice inicial el árbol de expansión crece, añadiendo a W, en cada pasada otro vértice z todavía no incluido en W, tal que si u es un vértice cualquiera de W, la arista (u,z) es la más corta, la de menor coste. El proceso termina cuando todos los vértices del grafo están en W y, por consiguiente, el árbol de expansión con todos los vértices está formado, además es mínimo porque en cada pasada se ha añadido la menor arista. La Figura 18.25 muestra un grafo conectado, formado por 10 vértices. Inicialmente se incorpora al conjunto W el vértice 1, W = {1}. De todas las aristas que forma parte el vértice 1, la de menor coste es (1, 2), por ello se añade a W el vértice 2, W = {1, 2}. La siguiente arista mínima, formada por cualquier vértice de W y los vértices no incorporados es (2, 4), entonces se añade a W el vértice 4, W = {1, 2, 4}. La siguiente pasada añade el vértice 5, al ser la arista (4, 5) la mínima. Una nueva pasada incorpora el vértice 8 ya que la arista (5, 8) es la de menor coste, W = {1, 2, 4, 5, 8}. A continuación, se incorpora el vértice 7 ya que la arista (5, 7) es la de menor coste, W = {1, 2, 4, 5, 8, 7}; le siguen los vértices 9, 10, 3 y termina el algoritmo con el vértice 6, W = {1, 2, 4, 5, 8, 7, 9, 10, 3, 6}. La Figura 18.26 muestra el árbol de expansión de coste mínimo formado.

Grafos Representación de los Grafos

Algoritmo de Prim

El algoritmo de Prim es un algoritmo perteneciente a la teoría de los grafos para encontrar un árbol recubridor mínimo en un grafo conexo, no dirigido y cuyas aristas están etiquetadas.

En otras palabras, el algoritmo encuentra un subconjunto de aristas que forman un árbol con todos los vértices, donde el peso total de todas las aristas en el árbol es el mínimo posible. Si el grafo no es conexo, entonces el algoritmo encontrará el árbol recubridor mínimo para uno de los componentes conexos que forman dicho grafo no conexo.
El algoritmo incrementa continuamente el tamaño de un árbol, comenzando por un vértice inicial al que se le van agregando sucesivamente vértices cuya distancia a los anteriores es mínima. Esto significa que en cada paso, las aristas a considerar son aquellas que inciden en vértices que ya pertenecen al árbol.

El árbol recubridor mínimo está completamente construido cuando no quedan más vértices por agregar.

Grafos Representación de los Grafos

Algoritmo de Prim

El algoritmo podría ser informalmente descrito siguiendo los siguientes pasos:

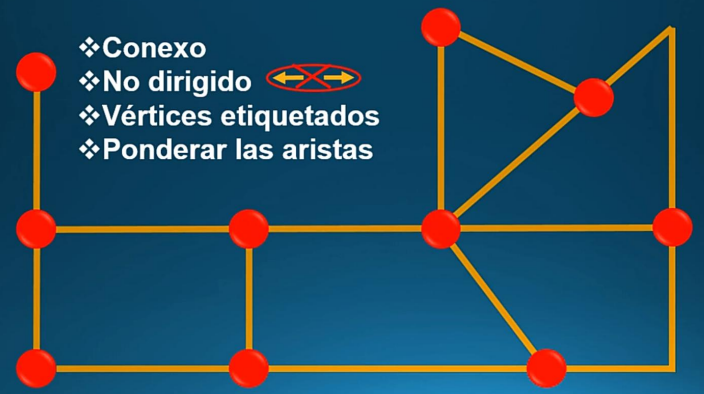
1. Inicializar un árbol con un único vértice, elegido arbitrariamente, del grafo.
2. Aumentar el árbol por un lado. Llamamos lado a la unión entre dos vértices: de las posibles uniones que pueden conectar el árbol a los vértices que no están aún en el árbol, encontrar el lado de menor distancia y unirlo al árbol.
3. Repetir el paso 2 (hasta que todos los vértices pertenezcan al árbol)

Como se ha descrito arriba, el vértice inicial para el algoritmo será elegido arbitrariamente, porque la primera iteración del bucle principal del algoritmo tendrá un número de vértices en Q que tendrán todos el mismo tamaño, y el algoritmo empezará automáticamente un nuevo árbol en F cuando complete un árbol de expansión a partir de cada vértice conectado del grafo. El algoritmo debe ser modificado para empezar con cualquier vértice particular s para configurar C[s] para que sea un número más pequeño que los otros valores de C (por norma, cero), y debe ser modificado para solo encontrar un único árbol de expansión y no un bosque entero de expansión, parando cuando encuentre otro vértice con "flag" que no tiene ningún lado asociado.

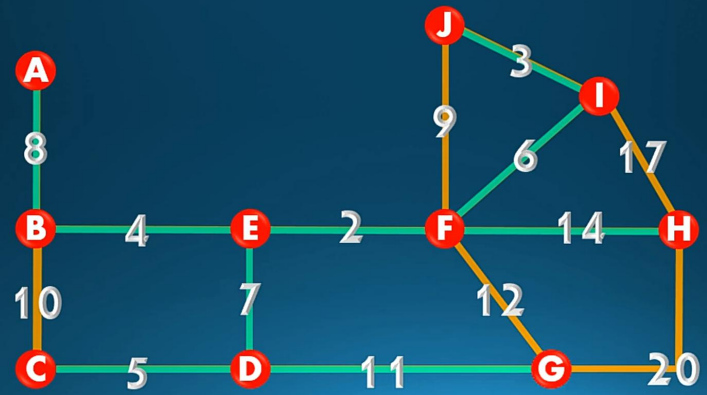
Hay diferentes variaciones del algoritmo que difieren unas de otras en cómo implementar Q: Como una única Lista enlazada o un vector de vértices, o como una estructura de datos organizada con una cola de prioridades, más compleja. Esta elección lidera las diferencias en complejidad de tiempo del algoritmo. En general, una cola de prioridades será más rápida encontrando el vértice v con el mínimo coste, pero ello conllevará actualizaciones más costosas cuando el valor de C[w] cambie.

ALGORITMO DE PRIM

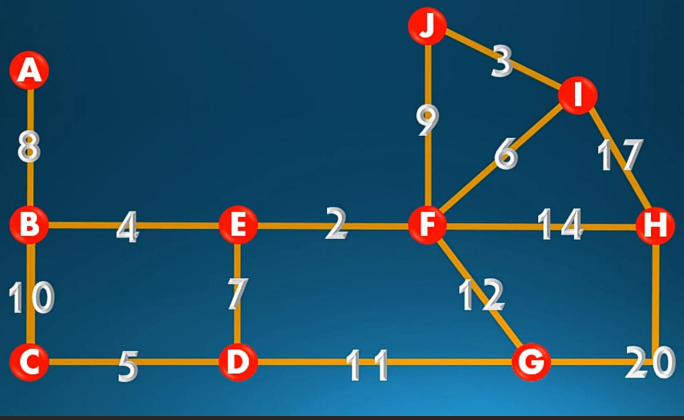
- Conexo
- No dirigido
- Vértices etiquetados
- Ponderar las aristas



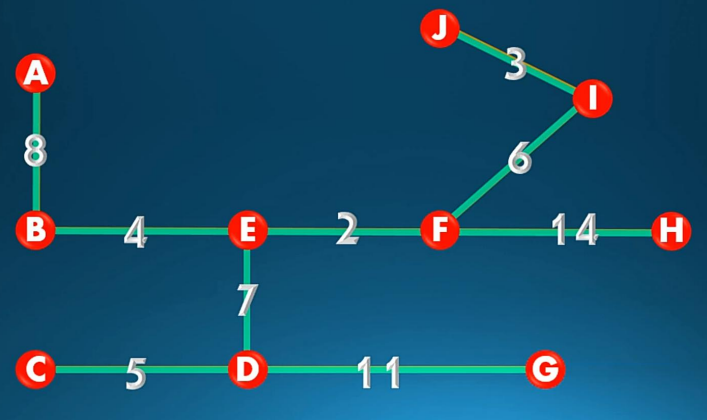
ALGORITMO DE PRIM



ALGORITMO DE PRIM



ALGORITMO DE PRIM



Grafos | Graph

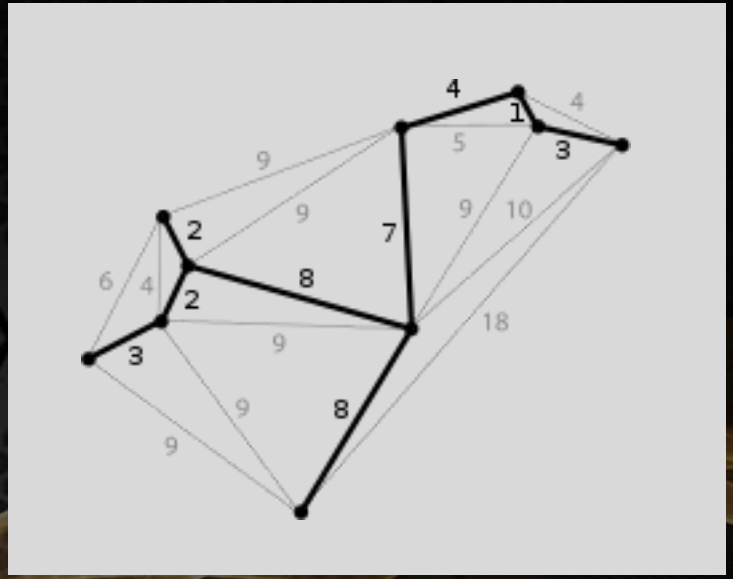
ÁRBOL DE EXPANSIÓN DE COSTE MÍNIMO

12

Algoritmo de Kruscal

Grafos Representación de los Grafos

Algoritmo de Kruscal



El algoritmo de Kruskal es un algoritmo de la teoría de grafos para encontrar un árbol recubridor mínimo en un grafo conexo y ponderado. Es decir, busca un subconjunto de aristas que, formando un árbol, incluyen todos los vértices y donde el valor de la suma de todas las aristas del árbol es el mínimo. Si el grafo no es conexo, entonces busca un bosque expandido mínimo (un árbol expandido mínimo para cada componente conexa). Este algoritmo toma su nombre de Joseph Kruskal, quien lo publicó por primera vez en 1956

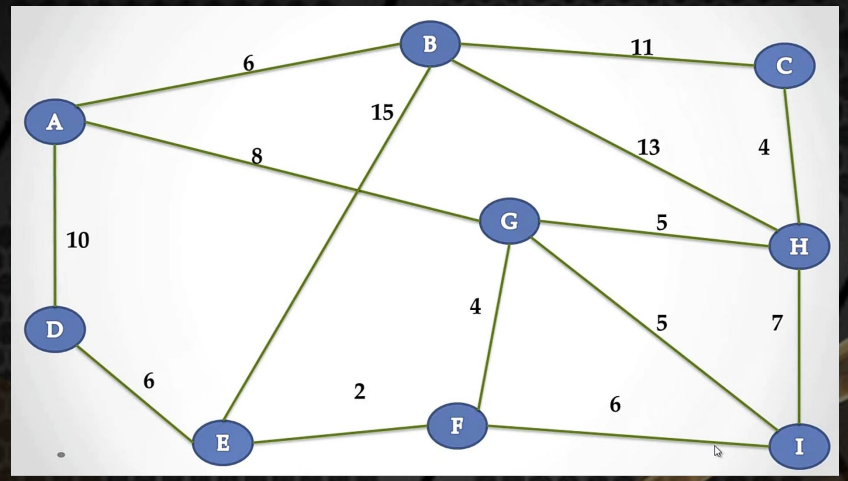
Kruskal propone otra estrategia para encontrar el árbol de expansión de coste mínimo. El árbol se empieza a construir con todos los vértices del grafo G pero sin aristas. Cada uno de estos vértices forma una componente conexa de un solo vértice. El algoritmo construye componente conexas de mayor tamaño hasta conseguir que todos los vértices del grafo se encuentren en una misma componte conexa. En cada iteración examina las aristas del grafo en orden creciente de peso formando una nueva componente conexa al unir dos componentes conexas distintas por una nueva arista del gafo de coste mínimo. Si la arista de coste mínimo conecta vértices que se encuentran en dos componentes conexas distintas, entonces se añade la arista al árbol de expansión T y se forma una nueva componente conexa eliminado las dos anteriores; en caso contrario se rechaza ya que daría lugar a un ciclo (no sería árbol). Cuando todos los vértices están en un sólo componente, T, éste es el árbol de expansión de coste mínimo del grafo G. El algoritmo de Kruskal asegura que el árbol no tiene ciclos ya que para añadir una arista sus vértices deben estar en dos componentes distintas y, además, es de coste mínimo ya que examina las aristas en orden creciente de sus pesos, por lo que necesariamente obtiene el árbol de expansión de coste mínimo

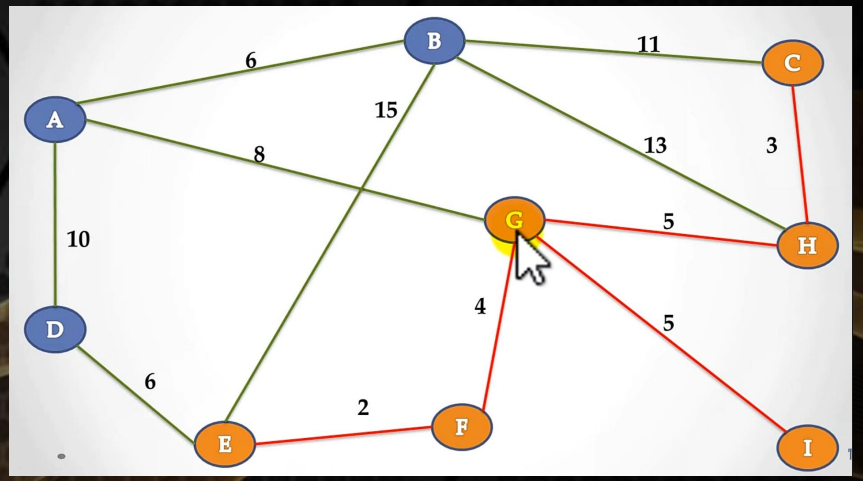
Grafos Representación de los Grafos

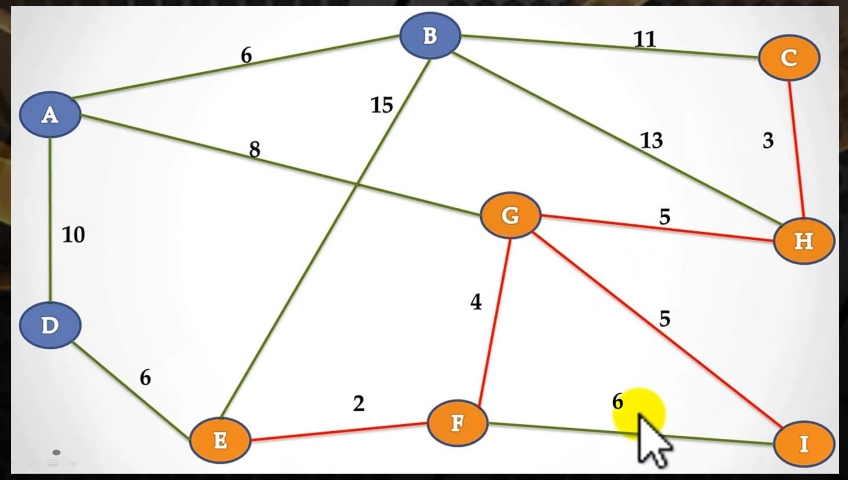
Algoritmo de Kruscal

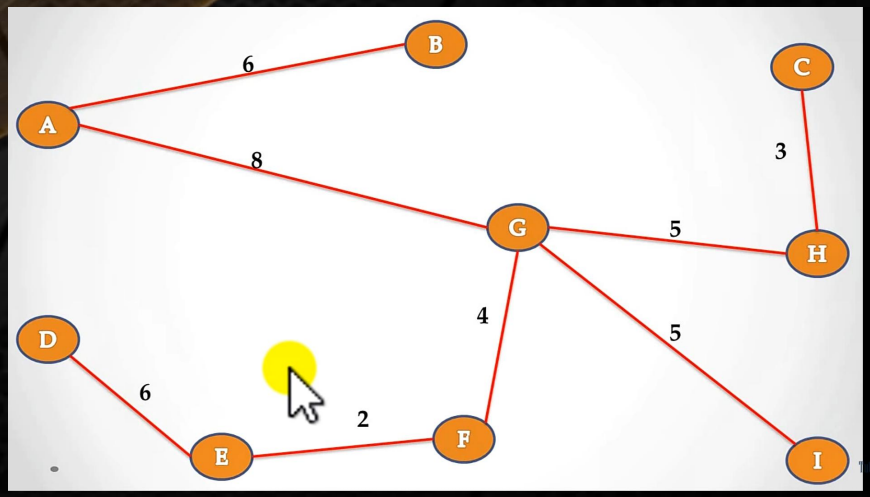
Proceso:

1. Seleccione el Arco de Menor Longitud.
2. En cada Iteración Agregue e Arco de Menor Longitud del conjunto restante
3. El algoritmo finaliza cuando todos los nodos estén conectados.









Grafos Representación de los Grafos

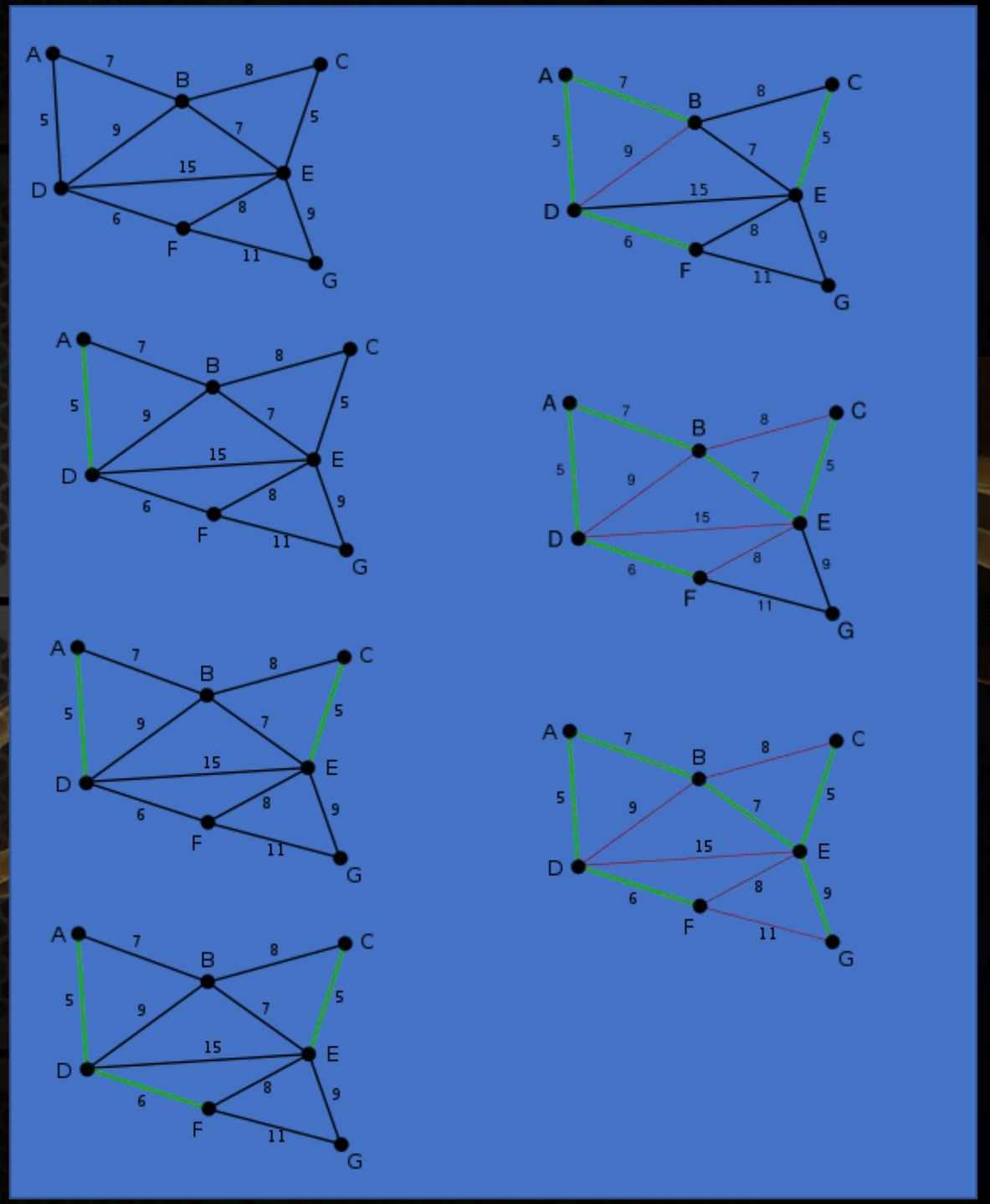
Algoritmo de Kruscal

Descripción
El algoritmo de Kruskal es un ejemplo de algoritmo voraz que funciona de la siguiente manera:

- se crea un bosque B (un conjunto de árboles), donde cada vértice del grafo es un árbol separado
- se crea un conjunto C que contenga a todas las aristas del grafo
- mientras C es no vacío
- eliminar una arista de peso mínimo de C
- si esa arista conecta dos árboles diferentes se añade al bosque, combinando los dos árboles en un solo árbol
- en caso contrario, se desecha la arista

Al acabar el algoritmo, el bosque tiene un solo componente, el cual forma un árbol de expansión mínimo del grafo.

En un árbol de expansión mínimo se cumple:
la cantidad de aristas del árbol es la cantidad de nodos menos uno (1).



X Classrrom GRAPH LABS

Laboratorio de Grafos

Muchas Gracias.